


# Análisis y Diseño del Sistema Transaccional de Comercio Electrónico Nalu

Mario Mendoza Huertas

Departamento de Ingeniería, Programa de Ingeniería de Software

Universidad Manuela Beltrán

Sistemas Transaccionales, Grupo 9

Ing. Fernando Benavides

20 de septiembre de 2026

**Resumen**

Nalu es un emprendimiento de joyería que comercializa su joyería bajo un modelo de venta por pedido, en el que cada venta depende si el cliente decide contactar directamente con el propietario. Esta modalidad mantiene la operación detenida cuando no hay contacto entrante y no deja registro estructurado de productos, existencias, clientes ni pedidos. Es así que el presente documento desarrolla el análisis y diseño de un sistema transaccional de comercio electrónico que estructura los procesos de publicación de catálogo, control de inventario, registro de pedidos, pagos y envío del emprendimiento Nalu. Definiendo la problemática y la necesidad del negocio, se establece un objetivo central con cinco objetivos específicos y se documenta la solución mediante una arquitectura de software en tres capas, diagramas de casos de uso, un diagrama de clases y un modelo relacional. El diseño contempla la ampliación del catálogo, que se proyecta en piezas exclusivas de plata a productos en acero, plata, oro, esmeraldas y cuarzo, por lo que el modelo de datos incorpora una estructura de materiales y variantes capaz de representar la ley del metal, el peso, la talla y las piedras de cada pieza. La solución se plantea como un producto mínimo viable que integra catálogo, categorías, fichas de producto, carrito, proceso de compra, gestión de pedidos e inventario y panel administrativo, junto con la gestión de contenido y metadatos que permite que el catálogo sea interpretado por motores de búsqueda y por sistemas basados en inteligencia artificial.

Palabras clave: comercio electrónico, sistema transaccional, análisis y diseño, UML, modelo relacional, SEO, GEO, joyería.

**Abstract**

Nalu is a jewelry business that sells its pieces through a made-to-order model in which every sale depends on the customer contacting the owner directly. This approach halts the operation whenever no inquiries come in and leaves no structured record of products, stock, customers, or orders. This document develops the analysis and design of a transactional e-commerce system that structures the business processes of catalog publishing, inventory control, order registration, payment, and shipping. It defines the business problem and need, establishes a central objective with five specific objectives, and documents the solution through a three-tier software architecture, use case diagrams, a class diagram, and a relational data model. The design supports the planned catalog expansion from silver-only pieces to steel, silver, gold, emeralds, and quartz, so the data model includes a material and variant structure able to represent metal purity, weight, size, and the gemstones of each piece. The solution is proposed as a minimum viable product covering catalog, categories, product pages, shopping cart, checkout, order and inventory management, and an administrative dashboard, along with the content and metadata management that allows the catalog to be interpreted by search engines and artificial intelligence systems.

*Keywords: e-commerce, transactional system, analysis and design, UML, relational model, SEO, GEO, jewelry.*

## Contenido

**Resumen** **2**
**Abstract** **3**
**Contenido** **4**
**Introducción** **6**
**Justificación** **7**
**Problemática** **8**
**Necesidad** **9**
**Pregunta problema** **10**
**1. Información general del proyecto** **11**
1.1. Nombre del sistema 11
1.7. Alcance de la solución 11
**2. Objetivo central** **11**
**3. Objetivos específicos** **12**
**4. Arquitectura de software** **12**
4.1. Tipo de arquitectura seleccionada 13
4.2. Componentes del sistema y función de cada uno 15
4.3. Comunicación entre los componentes 16
4.4. Justificación de la arquitectura seleccionada 16
**5. Diagrama de casos de uso UML** **17**
5.1. Actores del sistema 18
5.2. Descripción de los principales casos de uso 21
**6. Diagrama de clases UML** **23**
6.1. Clases principales y responsabilidades 30
**7. Modelo relacional** **31**
7.1. Descripción de las tablas 34
7.2. Relaciones y cardinalidades 35
7.3. Consideraciones de integridad y normalización 36
**8. Integración de los modelos** **37**
**9. Requerimientos funcionales** **39**
**10. Requerimientos no funcionales** **43**
**11. Marco conceptual** **47**
11.1. Conceptos fundamentales del proyecto 47
11.2. Conceptos de sistemas transaccionales 48
11.3. Conceptos tecnológicos 50
11.4. Aplicación de los conceptos al proyecto 53
**12. Marco contextual** **54**

12.1. Contexto del proyecto 54
12.2. Problemática identificada 54
12.3. Usuarios y beneficiarios 55
12.4. Necesidad que se pretende atender 55
12.5. Descripción del producto o prototipo 56
12.6. Alcance del proyecto 57
**13. Marco integrador: incorporación del servicio de inteligencia artificial 58**
13.1. Servicio de inteligencia artificial propuesto 58
13.2. Función dentro del proyecto 58
13.3. Integración con el sistema 60
13.4. Resultado esperado 62
**14. Referencias 64**

## Introducción

El comercio electrónico dejó de ser un canal complementario para convertirse en una vía habitual de compra en Colombia. Según la Cámara Colombiana de Comercio Electrónico (CCCE, 2026), durante 2025 se registraron 684,6 millones de transacciones en línea por un valor de 145,4 billones de pesos, lo que representa un crecimiento del 19,9 % en el número de operaciones frente al año anterior. Para un emprendimiento pequeño, esta tendencia significa que el cliente potencial ya está habituado a buscar, comparar y comprar productos sin intermediación de una conversación previa, y que un negocio sin presencia transaccional en línea queda por fuera de ese comportamiento de compra.

Nalu es un emprendimiento de joyería pequeño que ha operado bajo un modelo de venta por pedido: la venta solo ocurre cuando el cliente toma la iniciativa de contactar al propietario. Este modelo dejó de generar ingresos de manera sostenida y, además, no ofrece ningún registro estructurado de productos, existencias,descripciones, clientes ni pedidos. A esto se suma una decisión de negocio reciente: ampliar el catálogo, que antes se limitaba a piezas en plata, hacía acero, oro, esmeraldas y cuarzo, lo que multiplica los atributos que deben administrarse por cada joya.

Basándose en la necesidad del negocio se propone el desarrollo de un producto de ecommerce para el emprendimiento, dando no solo mayor control en los procesos internos del emprendimiento sino también dando visibilidad de este, aumentando las probabilidades de venta o de compradores potenciales.

## Justificación

La justificación del proyecto se sustenta en tres argumentos: el comportamiento del mercado, el impacto sobre la operación del negocio y la pertinencia académica del ejercicio.

En primer lugar, el mercado al que se dirige el emprendimiento ya opera en línea. El comercio electrónico colombiano cerró 2025 con 684,6 millones de transacciones y ventas por 145,4 billones de pesos, con un crecimiento del 19,9 % en el número de operaciones respecto de 2024 (CCCE, 2026). Este dato indica que la frecuencia de compra digital aumenta y que el consumidor incorporó el canal de internet a sus hábitos de compra, de modo que un negocio que solo vende por contacto directo compite en desventaja frente a competidores que permiten comprar sin intermediación.

En segundo lugar, el impacto sobre la operación es directo y medible. Un sistema que publique el catálogo y registre los pedidos convierte un proceso hoy manual y dependiente de la disponibilidad del propietario en un proceso rápido y disponible de manera permanente. El registro sistemático de las operaciones permite conocer la rotación por material y categoría, identificar las piezas agotadas antes de comprometerse y sustentar las decisiones de reposición y de precio con información propia y no con percepciones. Al buscar ampliar el catálogo a acero, oro, esmeraldas y cuarzo, esta estructuración deja de ser conveniente y se vuelve indispensable, pues el número de combinaciones de material, talla y piedra que deben administrarse crece considerablemente.

Los beneficios esperados alcanzan a dos grupos. Al propietario del emprendimiento, que dispondrá de herramientas para gestionar productos, inventario, promociones, pedidos y clientes desde un único lugar. Y a los compradores, que contarán con un canal de consulta y compra

disponible de forma permanente, con información detallada y actualizada de cada pieza, sin depender de que haya alguien disponible para responder.

En tercer lugar, el proyecto constituye un escenario real para aplicar el ciclo de análisis y diseño de software: la identificación de requisitos a partir de un problema de negocio, la especificación de funcionalidades mediante casos de uso, el diseño de la estructura estática del sistema y la definición del modelo de datos que garantiza la integridad de las transacciones (Sommerville, 2011; Pressman & Maxim, 2021). El resultado no es únicamente un ejercicio académico: es el insumo técnico con el que el emprendimiento podrá construir la solución.

## Problemática

Nalu comercializa joyería bajo un modelo de venta por pedido: el cliente interesado contacta directamente al propietario por mensajería o en persona, consulta precio y disponibilidad, y la pieza se entrega o se consigue después de esa solicitud. En este esquema, la totalidad de los ingresos depende de que sea el cliente quien inicie el contacto; cuando ese contacto no se produce, la operación comercial se detiene por completo. Esta es precisamente la situación actual del emprendimiento: el volumen de ventas ha disminuido de forma considerable porque no existe ningún mecanismo medible mediante el cual el negocio capte demanda de manera constante.

El análisis de la operación permite identificar cuatro problemas concretos de negocio:

1. **Dependencia del contacto manual.** No existe un espacio en el que el cliente pueda ver el catálogo completo, verificar disponibilidad, conocer el precio y cerrar la compra por sí mismo. Cada venta consume tiempo del propietario en responder las mismas preguntas, y el horario de atención del negocio coincide con su disponibilidad personal.

2. **Ausencia de registro estructurado de la operación.** Los precios, las existencias, los pedidos, los datos de los clientes y los envíos se manejan en conversaciones dispersas y anotaciones informales. En consecuencia, no es posible determinar con precisión qué piezas se vendieron en un periodo, cuáles tienen mayor rotación, cuáles están agotadas o cuánto se facturó, de modo que las decisiones comerciales se toman sobre percepciones y no sobre información verificable.
3. **Crecimiento del catálogo sin una estructura que lo soporte.** El emprendimiento comercializaba exclusivamente piezas en plata y proyecta ampliar su oferta a acero, oro, esmeralda y cuarzo. Cada material incorpora atributos y reglas de precio distintas —ley o pureza, peso, certificación, talla, piedra—, por lo que administrar manualmente un catálogo multimaterial aumenta el riesgo de inconsistencias: piezas ofrecidas sin existencia, precios desactualizados o descripciones incompletas.
4. **Baja visibilidad de la oferta.** Al no existir fichas de producto publicadas de manera estructurada, la oferta del negocio no aparece cuando un comprador potencial busca una pieza específica, y el alcance se limita al círculo de contactos actuales del propietario.

En resumen, la problemática no radica en la ausencia de una herramienta tecnológica particular, sino en que los procesos comerciales de Nalu —publicación de la oferta, control de existencias, registro de pedidos, cobro y entrega— no están estructurados ni soportados por ningún sistema de información, lo que impide que el negocio venda de forma continua y que su propietario disponga de datos confiables para administrarlo.

## Necesidad

A partir de la problemática expuesta, el emprendimiento necesita un sistema transaccional que estructure y registre sus operaciones comerciales. De manera concreta, se requiere un catálogo publicado y consultable de forma permanente, con información completa y actualizada de cada pieza; un control de existencias que refleje la disponibilidad real por variante y evite comprometer piezas inexistentes; un proceso de pedido que registre qué se vendió, a quién, en qué condiciones y en qué estado se encuentra; un registro de pagos y envíos asociado a cada pedido; y una base de información que permita consultar el comportamiento de las ventas por periodo, material y categoría.

El emprendimiento requiere además un entorno administrativo desde el cual gestionar de manera estructurada el catálogo, los precios, la disponibilidad, las promociones, los pedidos y los clientes, reduciendo la dependencia de procesos manuales que consumen un tiempo considerable. Del lado del comprador, la necesidad es un espacio donde consultar toda la información de la pieza, verificar su disponibilidad en el momento y completar la compra en un proceso ordenado.

A esto se suma la necesidad de visibilidad: que las piezas y categorías del catálogo estén descritas de forma que puedan ser interpretadas y recuperadas tanto por motores de búsqueda como por herramientas basadas en inteligencia artificial, ampliando el alcance del negocio más allá de sus contactos actuales.

## Pregunta problema

¿Cómo estructurar los procesos de publicación de catálogo, control de inventario, registro de pedidos, pago y envío de la joyería Nalu en un sistema transaccional que le permita vender de forma permanente sin depender del contacto manual por pedido, y disponer de información confiable para administrar el negocio?

## 1. Información general del proyecto

### 1.1. Nombre del sistema

Nalu — Sistema transaccional de comercio electrónico para joyería.

### 1.7. Alcance de la solución

El alcance corresponde a un producto mínimo viable (MVP) que cubre el ciclo completo de una venta en línea. El sistema incluye:

- Gestión de catálogo: productos, categorías jerárquicas, materiales, variantes comerciales, imágenes y atributos de cada pieza.
- Control de inventario por variante, con registro de movimientos de entrada, salida y ajuste, y reserva de existencias durante el proceso de compra.
- Proceso de compra del cliente: consulta y filtrado del catálogo, ficha de producto, carrito, registro o autenticación de la cuenta y confirmación del pedido.
- Registro de pedidos con sus estados, detalle de las piezas vendidas, aplicación de promociones y consulta del estado por parte del cliente.
- Registro de pagos a través de una pasarela externa y registro de los envíos con su número de guía.

- Panel administrativo para la gestión de productos, inventario, pedidos, promociones, contenido y consulta de reportes de ventas.
- Administración del contenido descriptivo y de los metadatos de las páginas de producto y categoría.

El sistema no incluye, en esta etapa: la elaboración o el diseño de las piezas, la contabilidad y facturación electrónica ante la autoridad tributaria, la integración con marketplaces de terceros, un módulo de fidelización de clientes ni una aplicación móvil nativa.

## 2. Objetivo central

Documentar el análisis y el diseño de un sistema transaccional de comercio electrónico para la joyería Nalu que estructure y registre los procesos de publicación de catálogo, control de inventario, pedidos, pagos y envíos, de manera que el emprendimiento pueda pasar de un modelo de venta por pedido dependiente del contacto manual a un canal de venta permanente con información trazable para la toma de decisiones.

## 3. Objetivos específicos

1. Definir la estructura de información del catálogo multimaterial del emprendimiento —acero, plata, oro, esmeraldas y cuarzo—, incluyendo categorías, materiales, variantes comercializables y atributos técnicos de cada pieza, mediante el diagrama de clases y el modelo relacional.
2. Modelar el proceso transaccional de compra, desde la consulta del catálogo hasta la confirmación del pedido, el pago y el envío, identificando actores, casos de uso, precondiciones y estados asociados a cada operación.

3. Diseñar el mecanismo de control de inventario por variante, basado en el registro de movimientos y en la reserva de existencias durante el proceso de pedido, con el fin de mantener la disponibilidad publicada consistente con la disponibilidad real.
4. Establecer la arquitectura de software del sistema, identificando sus componentes, las responsabilidades de cada uno y los mecanismos de comunicación entre ellos, y justificando su pertinencia frente a las características del proyecto.
5. Especificar el modelo de datos relacional del sistema, incluyendo tablas, campos, claves primarias y foráneas, cardinalidades y las entidades de contenido y metadatos que soportan la visibilidad del catálogo, garantizando la integridad referencial de las transacciones.

## 4. Arquitectura de software

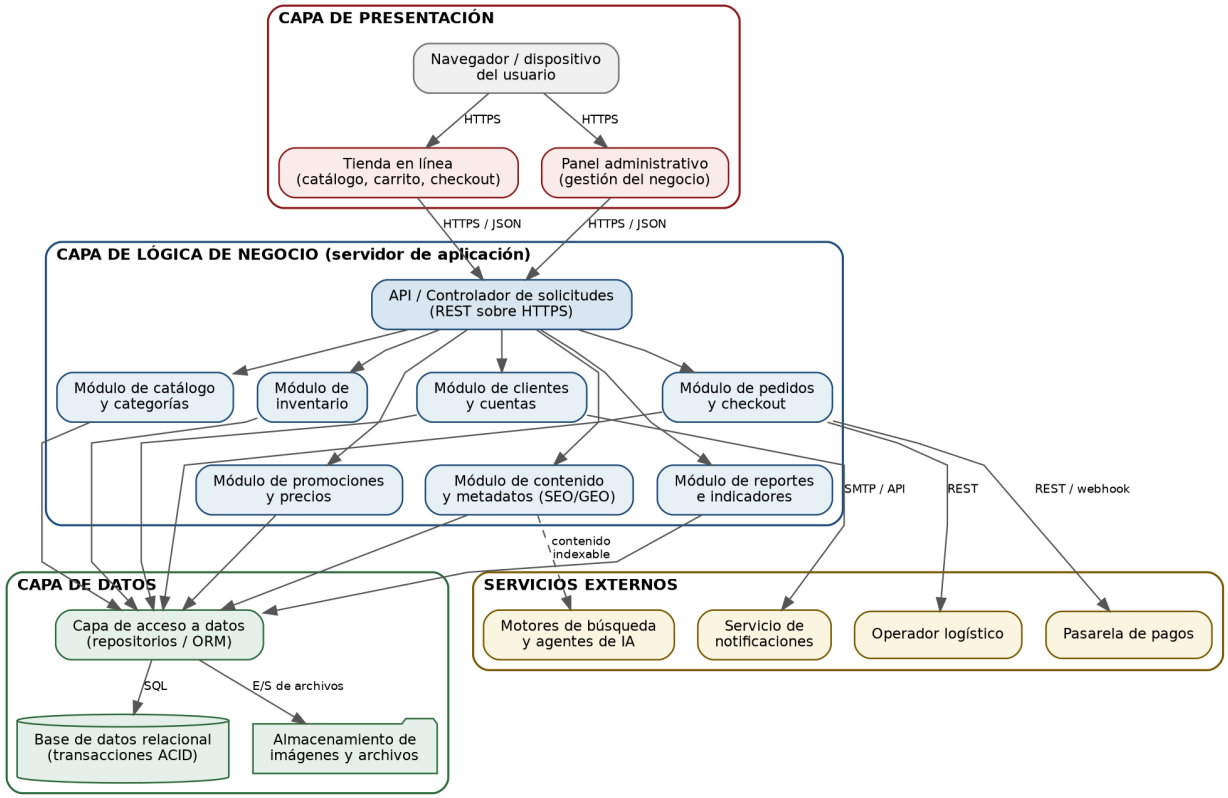
### 4.1. Tipo de arquitectura seleccionada

El sistema se estructura mediante una arquitectura en tres capas desplegada bajo el modelo cliente-servidor. Las capas corresponden a la presentación, la lógica de negocio y los datos, y cada una agrupa responsabilidades homogéneas: la capa de presentación construye y entrega las interfaces con las que interactúan el cliente final y la administrador; la capa de lógica de negocio concentra las reglas del dominio —cálculo de precios, validación de existencias, transiciones de estado del pedido, aplicación de promociones—; y la capa de datos resuelve la persistencia y la recuperación de la información. La comunicación entre capas es descendente: la presentación no accede directamente a la base de datos, sino que solicita operaciones a la capa de lógica de negocio, que es la única autorizada para invocar la capa de datos (Bass et al., 2021; Fowler, 2003).

Dentro de la capa de lógica de negocio, la funcionalidad se organiza en módulos cohesivos que se corresponden con los dominios identificados en los casos de uso: catálogo y categorías, inventario, pedidos y checkout, clientes, cuentas, promociones y precios, contenido y metadatos, y reportes e indicadores. Esta organización modular facilita que una funcionalidad se modifique sin afectar a las demás y hace explícita la correspondencia entre los modelos del documento y los componentes del sistema.

**Figura 1**

*Arquitectura en tres capas del sistema transaccional Nalu*




*Nota.* Las flechas indican la dirección de las solicitudes entre componentes. Los servicios externos se comunican con la capa de lógica de negocio mediante interfaces REST y notificaciones asincrónicas (webhooks). Elaboración propia.

### 4.2. Componentes del sistema y función de cada uno

La Tabla 1 describe los componentes que conforman cada capa, su responsabilidad dentro del sistema y los principales elementos del modelo de datos que administra.

**Tabla 1**
*Componentes de la arquitectura y su función*

| Capa | Componente | Función |
|---|---|---|
| Presentación | Tienda en línea | Entrega al cliente las páginas de catálogo, categoría, ficha de producto, carrito y proceso de compra. Renderiza el contenido y los metadatos administrados por el sistema. |
| Presentación | Panel administrativo | Interfaz de gestión para el propietario: productos, materiales, variantes, inventario, pedidos, promociones, contenido y reportes. |
| Lógica de negocio | API / controlador de solicitudes | Recibe las solicitudes de la capa de presentación, válida la autenticación y los permisos, y las enruta al módulo correspondiente. |
| Lógica de negocio | Módulo de catálogo y categorías | Administra productos, categorías, materiales, variantes e imágenes; calcula el precio de venta a partir del precio base y de los atributos del material. |
| Lógica de negocio | Módulo de inventario | Registra los movimientos de existencias, ejecuta la reserva y la liberación de unidades y expone la disponibilidad real de cada variante. |
| Lógica de negocio | Módulo de pedidos y checkout | Convierte el carrito en pedido, validar disponibilidad, calcula totales, controla las transiciones de estado y coordina el pago y el envío. |
| Lógica de negocio | Módulo de clientes y cuentas | Gestiona el registro, la autenticación, el perfil y las direcciones de los clientes, y envía las notificaciones asociadas al pedido. |
| Lógica de negocio | Módulo de promociones y precios | Administra los cupones y las reglas de descuento, valida su vigencia y calcula el descuento aplicable a un pedido. |
| Lógica de negocio | Módulo de contenido y metadatos | Administra las descripciones, los títulos, las URL canónicas y los datos estructurados de productos y categorías. |
| Lógica de negocio | Módulo de reportes e indicadores | Consolida la información de pedidos, productos e inventario para generar los reportes de ventas y rotación. |
| Datos | Capa de acceso a datos | Traduce las operaciones del dominio en instrucciones sobre la base de datos y administra las transacciones, aislando a la lógica de negocio del motor utilizado. |
| Datos | Base de datos relacional | Almacena de manera persistente las entidades del modelo relacional y garantiza la integridad referencial y las propiedades ACID de las transacciones. |
| Datos | Almacenamiento de archivos | Conserva las imágenes de los productos y los archivos asociados al catálogo. |
| Externos | Pasarela de pagos | Procesa la autorización y confirmación del pago y notifica el resultado de la transacción. |

| Capa | Componente | Función |
|---|---|---|
| Externos | Operador logístico | Recibe los datos del envío, genera la guía y reporta el estado de la entrega. |
| Externos | Servicio de notificaciones | Entrega al cliente los mensajes de confirmación y de cambio de estado del pedido. |

### 4.3. Comunicación entre los componentes

El navegador del usuario se comunica con la capa de presentación mediante el protocolo HTTPS. La capa de presentación, a su vez, consume los servicios de la capa de lógica de negocio a través de una interfaz REST que intercambia mensajes en formato JSON; en esa interfaz se validan la sesión del usuario y los permisos asociados a su rol antes de ejecutar cualquier operación.

Dentro de la capa de lógica de negocio, los módulos se invocan entre sí mediante llamadas internas: el módulo de pedidos solicita al de inventario la reserva de unidades, al de promociones la validación del cupón y al de clientes el envío de las notificaciones. Toda operación que requiera persistencia se dirige a la capa de acceso a datos, que la traduce en sentencias SQL sobre la base de datos relacional y agrupa en una única transacción las operaciones que deben confirmarse o revertirse en conjunto, como ocurre al crear un pedido y descontar su inventario.

La comunicación con los servicios externos se realiza mediante interfaces REST. En el caso del pago, el sistema envía la solicitud de autorización a la pasarela y recibe la confirmación por medio de una notificación asincrónica (webhook), lo que evita que el estado del pedido dependa de que el cliente permanezca en la página. En el caso del envío, el módulo de pedidos solicita al operador logístico la generación de la guía y almacena el número devuelto. Finalmente, los motores de búsqueda y los agentes basados en inteligencia artificial acceden

únicamente al contenido público de la tienda, sin ninguna interacción con las capas internas del sistema.

### 4.4. Justificación de la arquitectura seleccionada

La arquitectura en tres capas resulta adecuada para este proyecto por cuatro razones. La primera es la separación de responsabilidades: las reglas del negocio —cómo se calcula el precio de una pieza en oro, cuándo puede confirmarse un pedido, cómo se descuenta el inventario— quedan concentradas en una única capa y no se reproducen en la interfaz ni en la base de datos, lo que reduce la duplicación y facilita el mantenimiento (Bass et al., 2021).

La segunda es la consistencia transaccional. Las operaciones críticas del sistema comprometen varias entidades al mismo tiempo tanto confirmar un pedido implica crear el registro del pedido, su detalle, descontar el inventario de cada variante y registrar el pago. Al centralizar la persistencia en una única base de datos relacional gestionada por la capa de datos, estas operaciones se ejecutan como transacciones ACID, de modo que un fallo parcial no deja el inventario descuadrado (Elmasri & Navathe, 2016).

La tercera razón es la proporcionalidad frente al tamaño del proyecto. Una arquitectura de microservicios ofrecería independencia de despliegue, pero introduciría comunicación distribuida, consistencia eventual y una carga de operación que no se justifica en un emprendimiento con un único responsable técnico y un volumen inicial reducido de transacciones. La arquitectura en tres capas conserva la modularidad interna sin asumir esa complejidad, y permite una migración posterior si el volumen del negocio lo exige (Fowler, 2003).

La cuarta razón es la compatibilidad con los requisitos de visibilidad. Al generarse las páginas de catálogo desde el servidor, el contenido y los metadatos administrados por el sistema

quedan disponibles para los motores de búsqueda y los agentes de inteligencia artificial en la respuesta inicial, sin depender de procesos adicionales en el navegador del cliente (Enge et al., 2015).

## 5. Diagrama de casos de uso UML

El diagrama de casos de uso representa las funcionalidades del sistema desde la perspectiva de los actores que interactúan con él, delimitando qué hace el sistema sin describir cómo lo hace (Object Management Group [OMG], 2017; Booch et al., 2006). Dada la cantidad de funcionalidades identificadas, el modelo se presenta en dos vistas complementarias que comparten el mismo límite del sistema: los procesos de cara al cliente y los procesos de administración.

### 5.1. Actores del sistema

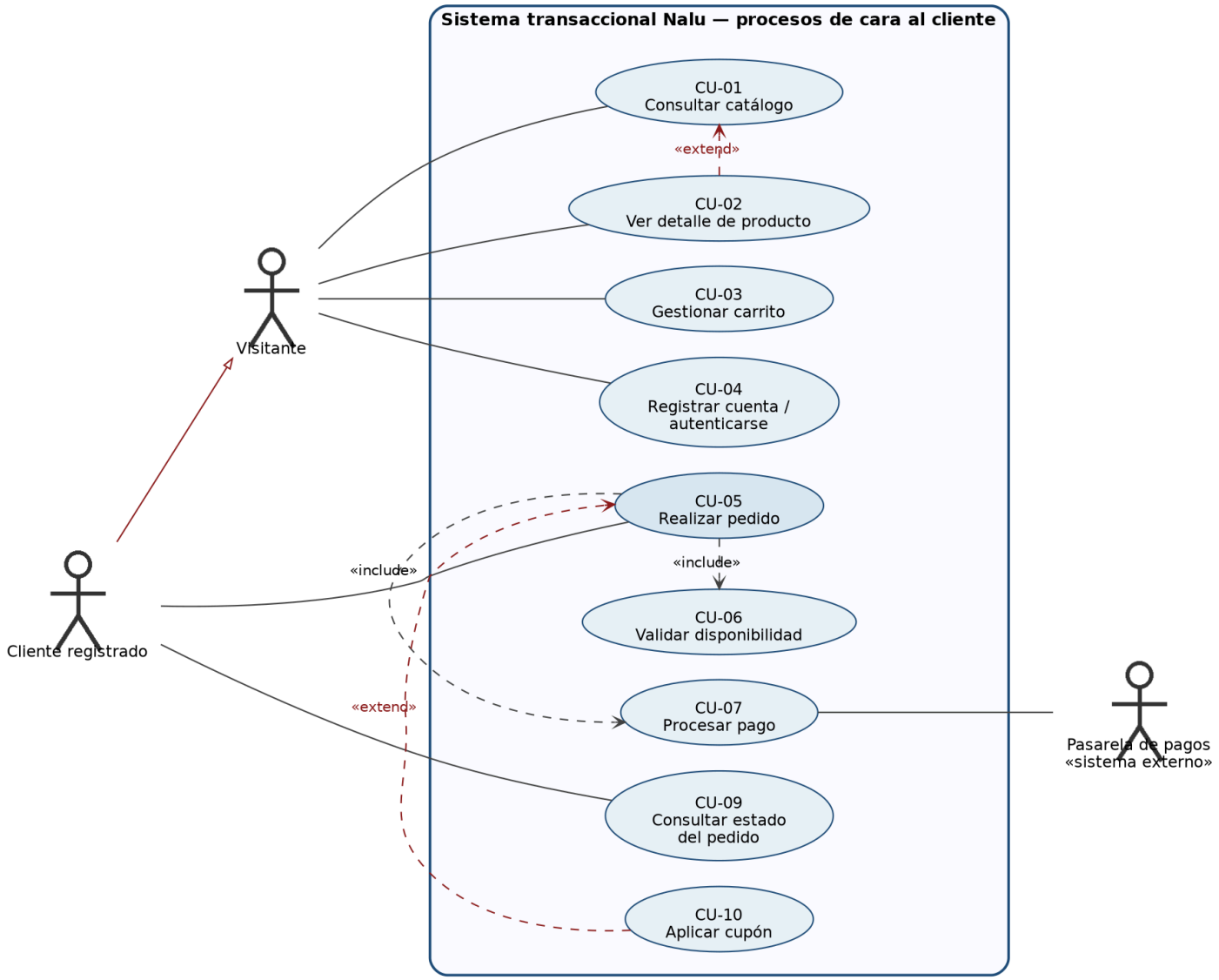
**Tabla 2**
*Actores identificados y su relación con el sistema*

| Actor | Tipo | Descripción |
|---|---|---|
| Visitante | Primario, humano | Usuario no autenticado que consulta el catálogo, revisa fichas de producto, arma un carrito y puede registrarse. |
| Cliente registrado | Primario, humano | Especialización del visitante. Tiene cuenta y direcciones registradas, confirma pedidos y consulta su estado. |
| Administrador | Primario, humano | Propietaria del emprendimiento. Gestiona catálogo, inventario, pedidos, promociones, contenido y reportes. |
| Pasarela de pagos | Secundario, sistema | Servicio externo que autoriza y confirma las transacciones de pago y notifica su resultado. |
| Operador logístico | Secundario, sistema | Servicio externo que genera la guía de envío y reporta el estado de la entrega. |

La relación entre el visitante y el cliente registrado es de generalización: el cliente hereda todas las funcionalidades del visitante y añade aquellas que requieren una cuenta autenticada.

**Figura 2**

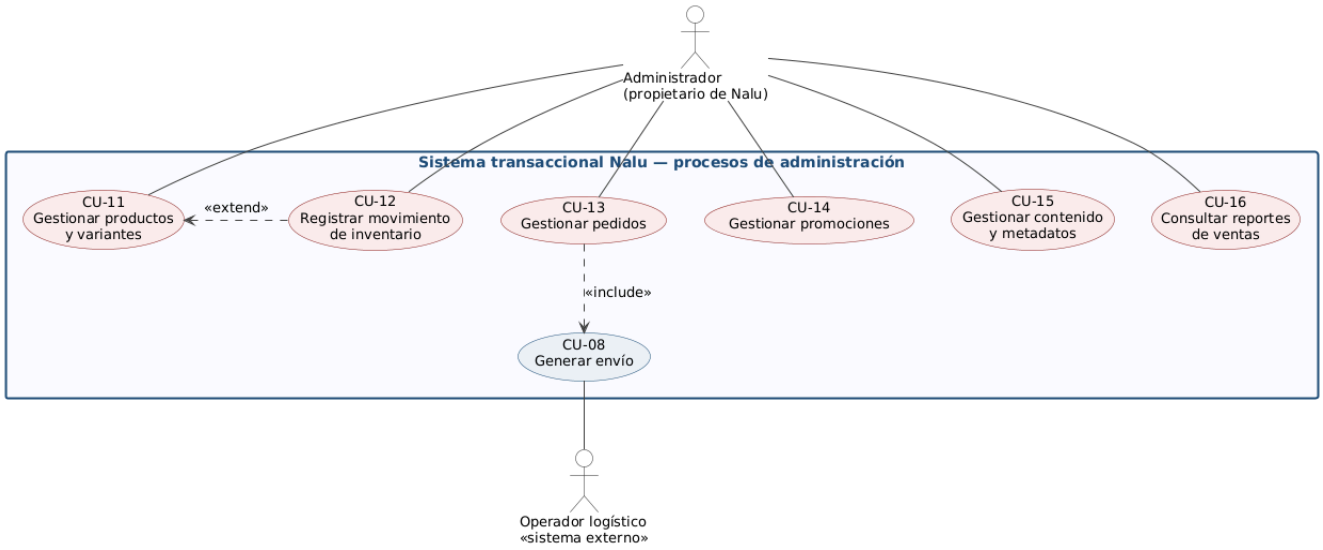
*Casos de uso de los procesos de cara al cliente*




*Nota.* Las relaciones «include» indican funcionalidades siempre ejecutadas como parte del caso de uso base; las relaciones «extend» indican comportamientos opcionales. Elaboración propia con base en OMG (2017).

**Figura 3**

*Casos de uso de los procesos de administración*




*Nota.* CU-08 se ejecuta como parte de la gestión del pedido y se comunica con el operador logístico. Elaboración propia con base en OMG (2017).

## 5.2. Descripción de los principales casos de uso

A continuación se describen los casos de uso más representativos del sistema, seleccionados por su impacto sobre el estado de la información y por su relación directa con los objetivos específicos del proyecto.

**Tabla 3**
*CU-01. Consultar catálogo*

| | |
|---|---|
| **Nombre** | CU-01. Consultar catálogo |
| **Actor principal** | Visitante (heredado por el cliente registrado) |
| **Objetivo** | Permitir que cualquier usuario explore las piezas disponibles y las filtre según material, categoría, rango de precio o disponibilidad. |
| **Precondiciones** | Existe al menos un producto publicado con una variante activa. El sistema se encuentra disponible. |
| **Flujo principal** | 1. El usuario ingresa a la tienda o a una categoría. 2. El sistema consulta los productos publicados y su disponibilidad. 3. El sistema presenta el listado con nombre, imagen principal, material y precio de venta. 4. El usuario aplica filtros por material, categoría o precio. 5. El sistema actualiza el listado con los productos que cumplen los criterios. |
| **Flujo alternativo** | 4a. Ningún producto cumple los criterios: el sistema informa la situación y sugiere retirar filtros. |
| **Resultado esperado** | El usuario visualiza un listado actualizado de productos disponibles, consistente con el inventario registrado. |

**Tabla 4**
*CU-03. Gestionar carrito*

| | |
|---|---|
| **Nombre** | CU-03. Gestionar carrito |
| **Actor principal** | Visitante o cliente registrado |
| **Objetivo** | Permitir que el usuario seleccione variantes de producto y defina las cantidades que desea adquirir antes de confirmar el pedido. |
| **Precondiciones** | La variante seleccionada se encuentra activa y tiene existencias disponibles. |
| **Flujo principal** | 1. El usuario elige una variante desde la ficha del producto e indica la cantidad. 2. El sistema verifica la disponibilidad de la variante. 3. El sistema crea el carrito si no existe y agrega el ítem con el precio vigente. 4. El sistema recalcula el subtotal. 5. El usuario puede modificar cantidades o eliminar ítems. |
| **Flujo alternativo** | 2a. La cantidad solicitada supera las existencias: el sistema informa la disponibilidad máxima y ajusta la cantidad. |
| **Resultado esperado** | El carrito queda registrado con los ítems, las cantidades y el subtotal actualizado. |

**Tabla 5**
*CU-05. Realizar pedido*

| | |
|---|---|
| **Nombre** | CU-05. Realizar pedido |
| **Actor principal** | Cliente registrado |
| **Objetivo** | Convertir el contenido del carrito en un pedido registrado, con su dirección de envío, sus totales y su pago asociado. |
| **Precondiciones** | El cliente está autenticado, su carrito contiene al menos un ítem y tiene registrada una dirección de envío. |
| **Flujo principal** | 1. El cliente inicia el proceso de compra. 2. El sistema ejecuta CU-06 para validar la disponibilidad de cada variante. 3. El cliente selecciona la dirección de envío y el sistema calcula el costo correspondiente. 4. El cliente aplica un cupón (CU-10, opcional) y el sistema recalcula el descuento. 5. El sistema crea el pedido en estado pendiente con su detalle y reserva las unidades. 6. El sistema ejecuta CU-07 para procesar el pago. 7. Confirmado el pago, el sistema cambia el pedido a estado pagado, descuenta el inventario reservado y notifica al cliente. |
| **Flujo alternativo** | 2a. Una variante ya no tiene existencias: el sistema informa la situación y solicita ajustar el carrito. 6a. El pago es rechazado: el pedido permanece pendiente, se liberan las unidades reservadas y se informa al cliente. |
| **Resultado esperado** | Queda registrado un pedido con su detalle, su dirección, sus totales, su pago y el inventario actualizado. |

**Tabla 6**
*CU-07. Procesar pago*

| | |
|---|---|
| **Nombre** | CU-07. Procesar pago |
| **Actor principal** | Cliente registrado. Actor secundario: pasarela de pagos |
| **Objetivo** | Registrar el cobro del pedido a través de la pasarela de pagos y dejar constancia del resultado de la transacción. |
| **Precondiciones** | Existe un pedido en estado pendiente con un total calculado. |
| **Flujo principal** | 1. El sistema crea el registro de pago asociado al pedido con estado iniciado. 2. El sistema envía a la pasarela el monto, la referencia del pedido y los datos requeridos. 3. El cliente completa la autenticación del medio de pago en la pasarela. 4. La pasarela notifica el resultado mediante un webhook. 5. El sistema actualiza el estado del pago y la referencia externa devuelta. |
| **Flujo alternativo** | 4a. La pasarela reporta rechazo o no responde: el pago queda en estado rechazado o pendiente de confirmación y el pedido no avanza a pagado. |
| **Resultado esperado** | El pedido cuenta con un registro de pago cuyo estado y referencia externa permiten su consolidación posterior. |

**Tabla 7**
*CU-11. Gestionar productos y variantes*

| | |
|---|---|
| **Nombre** | CU-11. Gestionar productos y variantes |
| **Actor principal** | Administrador |
| **Objetivo** | Crear y mantener las piezas del catálogo con sus materiales, variantes comercializables, imágenes y precios. |

| | |
|---|---|
| **Precondiciones** | El administrador está autenticado. Existen las categorías y los materiales requeridos. |
| **Flujo principal** | 1. El administrador registra el producto con su nombre, descripción, categoría y precio base. 2. Asocia los materiales de la pieza indicando el rol de cada uno —metal principal o piedra— y sus atributos de peso o quilates. 3. Registra las variantes comercializables con su talla, su SKU y su existencia inicial. 4. Carga las imágenes y define la principal. 5. Publica el producto. |
| **Flujo alternativo** | 3a. El SKU de la variante ya existe: el sistema rechaza el registro e informa la duplicidad. |
| **Resultado esperado** | El producto queda publicado en el catálogo con sus materiales, variantes, imágenes y disponibilidad inicial registrados. |

**Tabla 8**
*CU-12. Registrar movimiento de inventario*

| | |
|---|---|
| **Nombre** | CU-12. Registrar movimiento de inventario |
| **Actor principal** | Administrador |
| **Objetivo** | Actualizar las existencias de una variante dejando constancia del tipo, la cantidad y el motivo del movimiento. |
| **Precondiciones** | La variante existe en el catálogo y el administrador está autenticado. |
| **Flujo principal** | 1. El administrador selecciona la variante. 2. Indica el tipo de movimiento —entrada, salida o ajuste—, la cantidad y el motivo. 3. El sistema valida que la operación no genere existencias negativas. 4. El sistema registra el movimiento con su fecha y el usuario responsable. 5. El sistema actualiza la existencia de la variante. |
| **Flujo alternativo** | 3a. La salida supera la existencia disponible: el sistema rechaza el movimiento e informa el saldo actual. |
| **Resultado esperado** | La existencia de la variante queda actualizada y el movimiento registrado para su trazabilidad. |

**Tabla 9**
*CU-13. Gestionar pedidos*

| | |
|---|---|
| **Nombre** | CU-13. Gestionar pedidos |
| **Actor principal** | Administrador. Actor secundario: operador logístico |
| **Objetivo** | Hacer seguimiento a los pedidos recibidos y controlar su avance hasta la entrega. |
| **Precondiciones** | Existe al menos un pedido registrado y el administrador está autenticado. |
| **Flujo principal** | 1. El administrador consulta los pedidos filtrando por estado o fecha. 2. Selecciona un pedido y revisa su detalle, su pago y su dirección. 3. Cambia el estado del pedido según su avance. 4. Al despachar, el sistema ejecuta CU-08 y registra la transportadora y el número de guía. 5. El sistema notifica al cliente el cambio de estado. |
| **Flujo alternativo** | 3a. El pedido se cancela: el sistema revierte la reserva o reintegra las unidades al inventario y registra el motivo. |
| **Resultado esperado** | El pedido refleja su estado real y el cliente dispone de la información de seguimiento. |

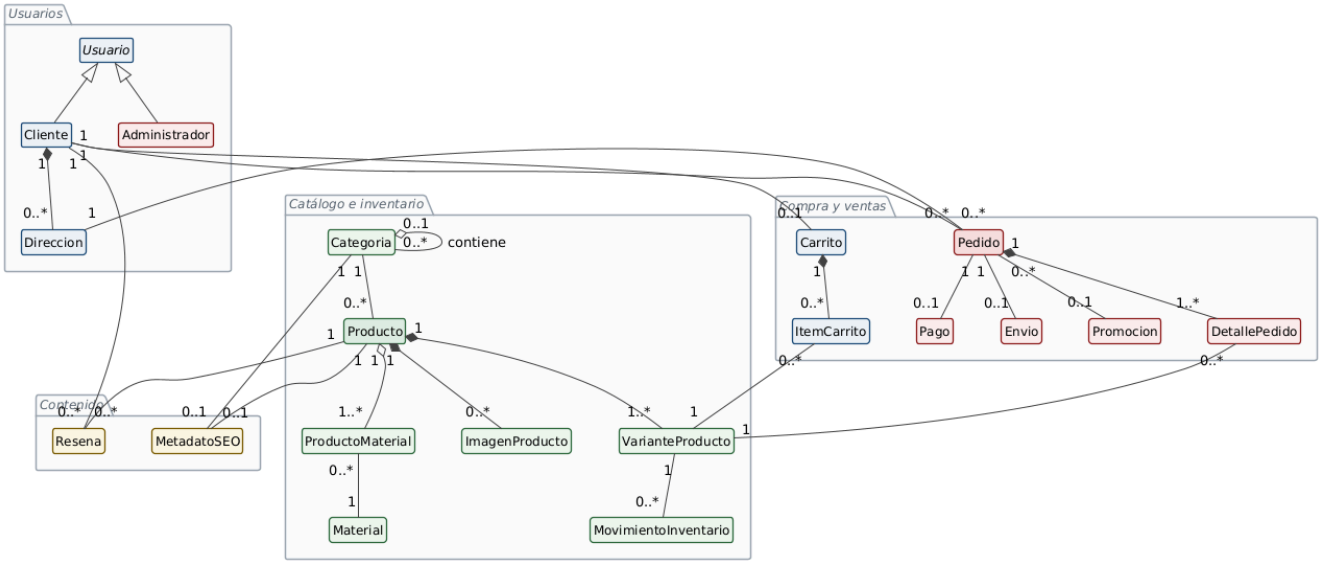
## 6. Diagrama de clases UML

El diagrama de clases representa la estructura estática del sistema: las clases que componen el dominio, sus atributos, sus operaciones y las relaciones entre ellas (Booch et al., 2006). Mientras los casos de uso describen qué debe hacer el sistema, el diagrama de clases describe con qué estructuras lo hace; por ello, cada caso de uso de la sección anterior encuentra aquí las clases que lo soportan.

Para preservar la legibilidad, el modelo se presenta primero en una vista general con todas las clases y sus relaciones, y luego en cuatro vistas detalladas que incluyen atributos y métodos. Las clases que aparecen en más de una vista, como VarianteProducto, son la misma clase del modelo y se repiten únicamente como punto de conexión entre vistas.

**Figura 4**

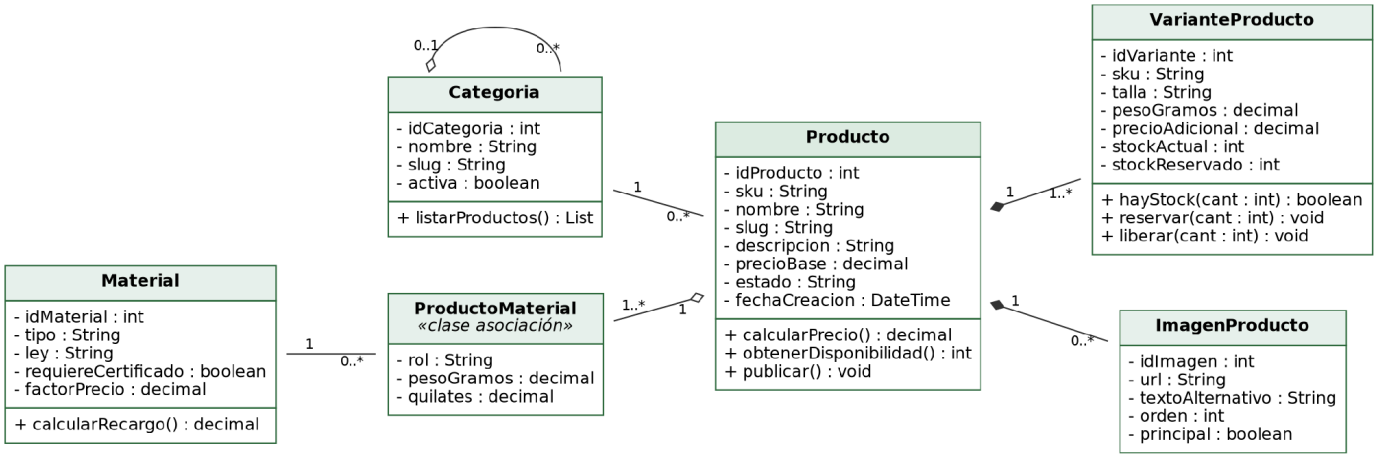
*Diagrama de clases. Vista general del modelo*




*Nota.* Los rombos rellenos indican composición; los rombos vacíos, agregación; el triángulo vacío, herencia. Las multiplicidades se indican en los extremos de cada relación. Elaboración propia.

**Figura 5**

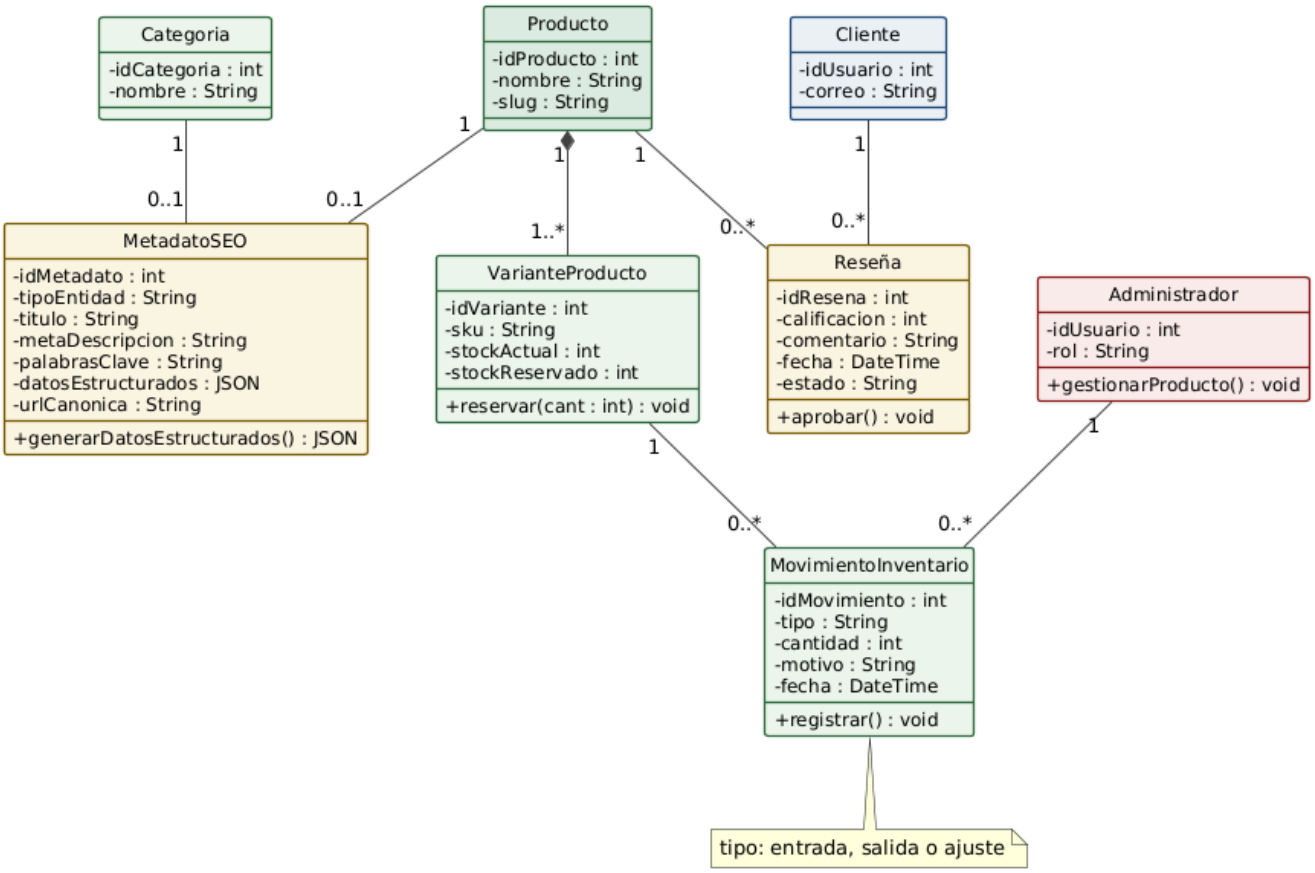
*Diagrama de clases. Vista de catálogo y productos*




*Nota.* ProductoMaterial es una clase de asociación que resuelve la relación muchos a muchos entre Producto y Material y almacena los atributos propios de esa relación. Elaboración propia.

**Figura 6**

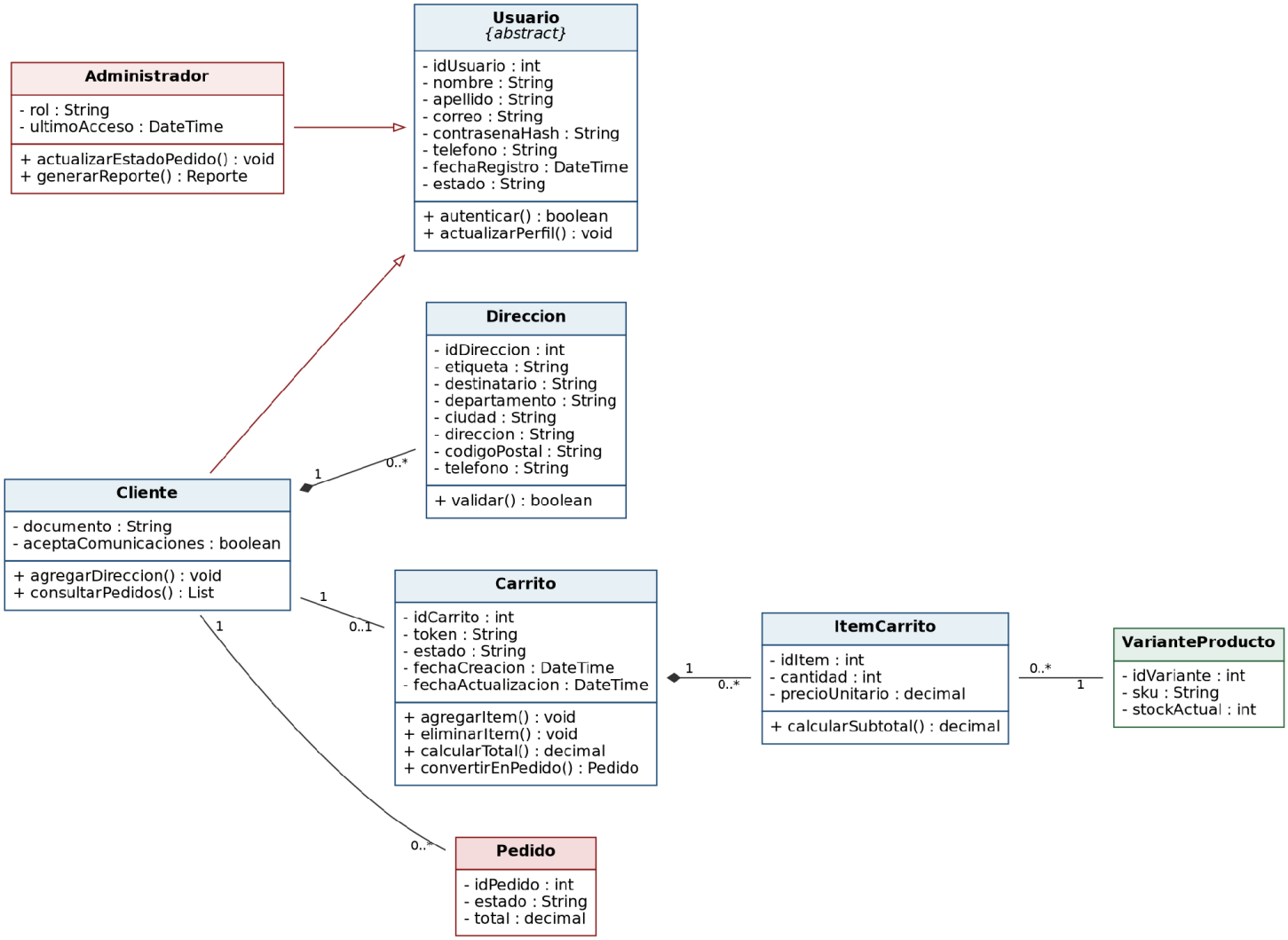
*Diagrama de clases. Vista de inventario, contenido y reseñas*




*Nota.* MovimientoInventario conserva la trazabilidad de cada cambio de existencias y el usuario que lo registró. Elaboración propia.

**Figura 7**

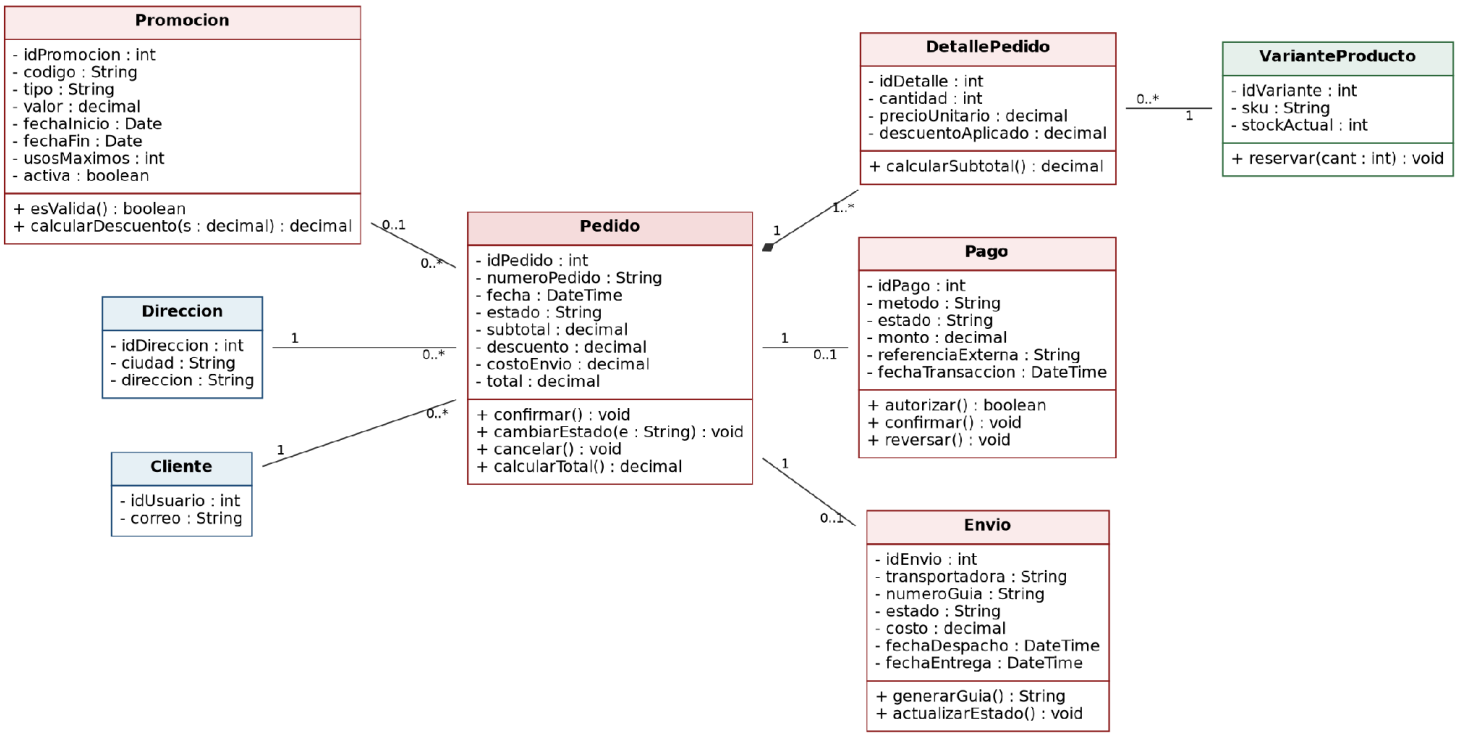
*Diagrama de clases. Vista de usuarios, direcciones y carrito*




*Nota.* Usuario es una clase abstracta especializada en Cliente y Administrador, lo que evita duplicar los atributos de autenticación. Elaboración propia.

**Figura 8**

*Diagrama de clases. Vista de pedido, pago, envío y promoción*




*Nota.* DetallePedido conserva el precio unitario al momento de la venta, de modo que un cambio posterior de precio no altera los pedidos históricos. Elaboración propia.

## 6.1. Clases principales y responsabilidades

**Tabla 10**
*Clases principales del modelo y su responsabilidad*

| Clase | Responsabilidad | Casos de uso |
|---|---|---|
| Usuario (abstracta) | Centraliza los datos de identificación y autenticación comunes a todos los usuarios del sistema. | CU-04 |
| Cliente | Representa al comprador: sus direcciones, su carrito, sus pedidos y sus reseñas. | CU-04, CU-05, CU-09 |
| Administrador | Representa a la propietaria del emprendimiento y su capacidad de gestión sobre el catálogo, el inventario y los pedidos. | CU-11 a CU-16 |
| Categoria | Organiza el catálogo en una estructura jerárquica mediante una relación reflexiva. | CU-01, CU-11 |
| Producto | Describe la pieza de joyería: su nombre, su descripción, su precio base y su estado de publicación. | CU-01, CU-02, CU-11 |
| Material | Define los materiales del catálogo —acero, plata, oro, esmeralda y cuarzo— con su ley y su factor de precio. | CU-01, CU-11 |
| ProductoMaterial | Clase asociación que registra el rol de cada material en una pieza y sus atributos de peso o quilates. | CU-11 |
| VarianteProducto | Unidad comercializable del producto. Concentra el SKU, la talla y las existencias, y resuelve la disponibilidad. | CU-03, CU-05, CU-06, CU-12 |
| MovimientoInventario | Registra cada cambio de existencias con su tipo, cantidad, motivo, fecha y responsable. | CU-12 |
| Carrito / ItemCarrito | Mantienen la selección temporal del usuario antes de confirmar el pedido. | CU-03 |
| Pedido / DetallePedido | Registran la transacción de venta, sus totales, su estado y las piezas vendidas con su precio histórico. | CU-05, CU-09, CU-13 |
| Pago | Conserva el resultado de la transacción con la pasarela y su referencia externa para conciliación. | CU-07 |
| Envio | Registra la transportadora, el número de guía y el estado de la entrega. | CU-08, CU-13 |
| Promocion | Define los cupones y las reglas de descuento, su vigencia y el cálculo del valor descontado. | CU-10, CU-14 |
| MetadatoSEO | Administra el título, la descripción, la URL canónica y los datos estructurados de productos y categorías. | CU-15 |

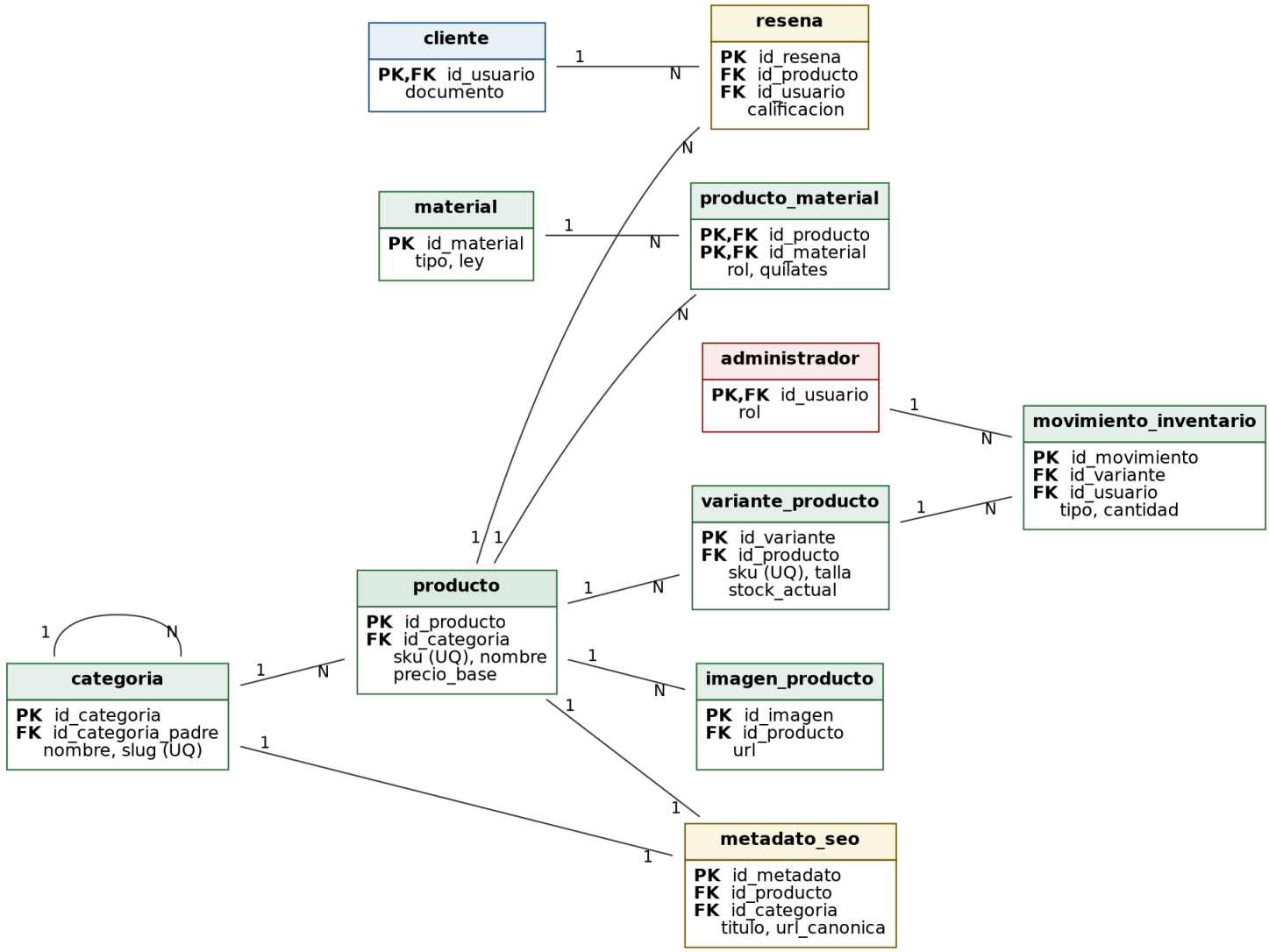
| Clase | Responsabilidad | Casos de uso |
|---|---|---|
| Resena | Registra la valoración y el comentario de un cliente sobre un producto adquirido. | CU-02 |

## 7. Modelo relacional

El modelo relacional traduce las clases del apartado anterior en tablas, atributos y restricciones de integridad. La correspondencia es directa: cada clase del dominio origina una tabla; las relaciones uno a muchos se implementan mediante claves foráneas en el lado "muchos"; las relaciones muchos a muchos se resuelven con tablas intermedias; y la herencia entre Usuario, Cliente y Administrador se implementa mediante tablas especializadas que comparten la clave primaria de la tabla general (Elmasri & Navathe, 2016; Date, 2001).

**Figura 9**

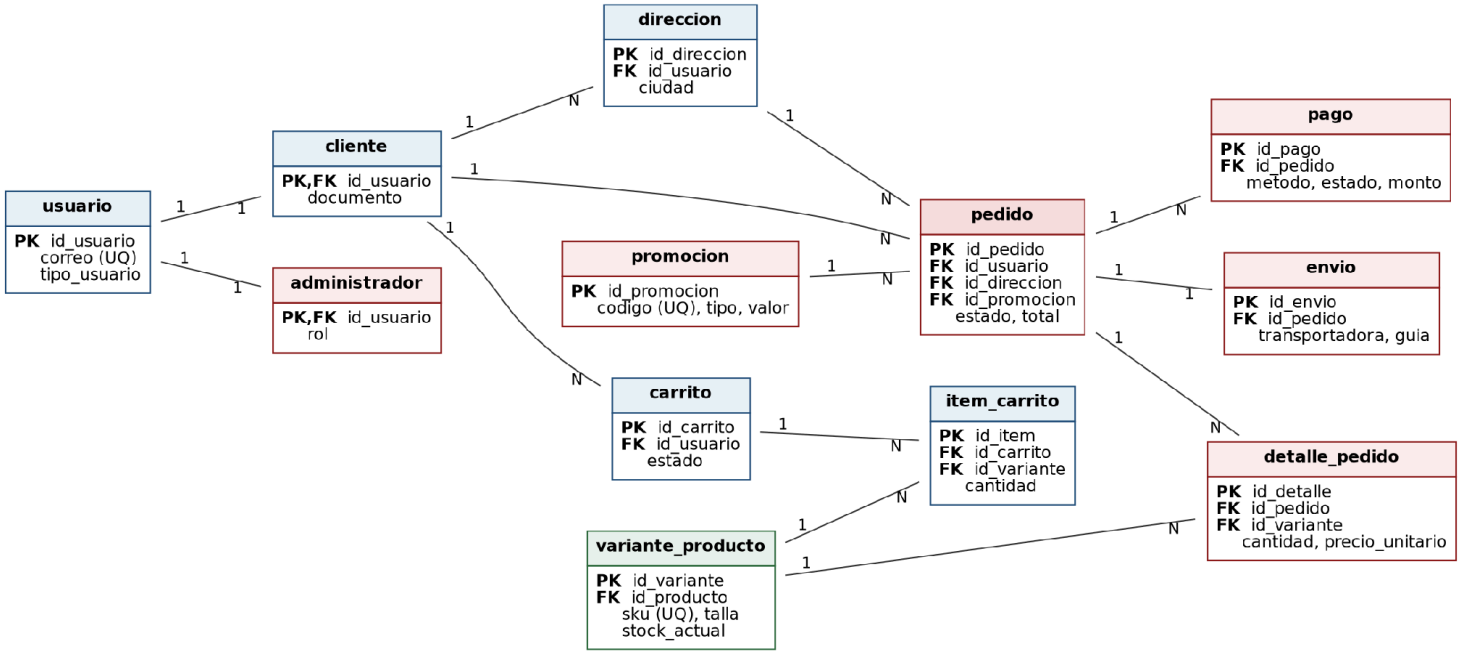
*Modelo relacional. Catálogo, inventario y contenido*




*Nota.* Se indican las claves primarias (PK), las claves foráneas (FK) y los atributos con restricción de unicidad (UQ). Elaboración propia.

**Figura 10**

*Modelo relacional. Usuarios, carrito y ventas*




*Nota.* Las tablas cliente y administrador comparten la clave primaria de usuario, implementando la herencia del diagrama de clases. Elaboración propia.

### 7.1. Descripción de las tablas

Las Tablas 11 y 12 detallan la estructura de cada tabla del modelo, indicando sus campos principales, sus tipos de datos y sus restricciones.

**Tabla 11**
*Tablas del subsistema de catálogo, inventario y contenido*

| Tabla | Campos y tipos de datos | Claves |
|---|---|---|
| categoria | id_categoria INT; id_categoria_padre INT NULL; nombre VARCHAR(80); slug VARCHAR(100) UQ; descripcion TEXT; activa BOOLEAN | PK: id_categoria. FK: id_categoria_padre → categoria |
| material | id_material INT; tipo VARCHAR(30); ley VARCHAR(20); requiere_certificado BOOLEAN; factor_precio DECIMAL(6,3) | PK: id_material |
| producto | id_producto INT; id_categoria INT; sku VARCHAR(40) UQ; nombre VARCHAR(120); slug VARCHAR(140) UQ; descripcion TEXT; precio_base DECIMAL(12,2); estado VARCHAR(20); fecha_creacion TIMESTAMP | PK: id_producto. FK: id_categoria → categoria |
| producto_material | id_producto INT; id_material INT; rol VARCHAR(20); peso_gramos DECIMAL(8,2); quilates DECIMAL(6,2) NULL | PK compuesta: (id_producto, id_material). FK: ambos campos |
| variante_producto | id_variante INT; id_producto INT; sku VARCHAR(40) UQ; talla VARCHAR(20); peso_gramos DECIMAL(8,2); precio_adicional DECIMAL(12,2); stock_actual INT; stock_reservado INT | PK: id_variante. FK: id_producto → producto |
| imagen_producto | id_imagen INT; id_producto INT; url VARCHAR(255); texto_alternativo VARCHAR(160); orden INT; principal BOOLEAN | PK: id_imagen. FK: id_producto → producto |
| movimiento_inventario | id_movimiento INT; id_variante INT; id_usuario INT; tipo VARCHAR(20); cantidad INT; motivo VARCHAR(160); fecha TIMESTAMP | PK: id_movimiento. FK: id_variante → variante_producto; id_usuario → administrador |
| metadato_seo | id_metadato INT; id_producto INT NULL; id_categoria INT NULL; titulo VARCHAR(160); meta_descripcion VARCHAR(320); palabras_clave VARCHAR(255); datos_estructurados JSON; url_canonica VARCHAR(255) | PK: id_metadato. FK: id_producto → producto; id_categoria → categoria |
| resena | id_resena INT; id_producto INT; id_usuario INT; calificacion INT; comentario TEXT; fecha TIMESTAMP; estado VARCHAR(20) | PK: id_resena. FK: id_producto → producto; id_usuario → cliente |

**Tabla 12**
*Tablas del subsistema de usuarios y ventas*

| Tabla | Campos y tipos de datos | Claves |
|---|---|---|
| usuario | id_usuario INT; nombre VARCHAR(60); apellido VARCHAR(60); correo VARCHAR(120) UQ; contrasena_hash VARCHAR(255); telefono VARCHAR(20); tipo_usuario VARCHAR(20); fecha_registro TIMESTAMP; estado VARCHAR(20) | PK: id_usuario |
| cliente | id_usuario INT; documento VARCHAR(20); acepta_comunicaciones BOOLEAN | PK y FK: id_usuario → usuario |
| administrador | id_usuario INT; rol VARCHAR(30); ultimo_acceso TIMESTAMP | PK y FK: id_usuario → usuario |
| direccion | id_direccion INT; id_usuario INT; etiqueta VARCHAR(40); destinatario VARCHAR(120); departamento VARCHAR(60); ciudad VARCHAR(60); direccion VARCHAR(160); codigo_postal VARCHAR(12); telefono VARCHAR(20) | PK: id_direccion. FK: id_usuario → cliente |
| carrito | id_carrito INT; id_usuario INT NULL; token VARCHAR(64); estado VARCHAR(20); fecha_creacion TIMESTAMP; fecha_actualizacion TIMESTAMP | PK: id_carrito. FK: id_usuario → cliente |
| item_carrito | id_item INT; id_carrito INT; id_variante INT; cantidad INT; precio_unitario DECIMAL(12,2) | PK: id_item. FK: id_carrito → carrito; id_variante → variante_producto |
| pedido | id_pedido INT; numero_pedido VARCHAR(20) UQ; id_usuario INT; id_direccion INT; id_promocion INT NULL; fecha TIMESTAMP; estado VARCHAR(25); subtotal DECIMAL(12,2); descuento DECIMAL(12,2); costo_envio DECIMAL(12,2); total DECIMAL(12,2) | PK: id_pedido. FK: id_usuario → cliente; id_direccion → direccion; id_promocion → promocion |
| detalle_pedido | id_detalle INT; id_pedido INT; id_variante INT; cantidad INT; precio_unitario DECIMAL(12,2); descuento_aplicado DECIMAL(12,2) | PK: id_detalle. FK: id_pedido → pedido; id_variante → variante_producto |
| pago | id_pago INT; id_pedido INT; metodo VARCHAR(30); estado VARCHAR(25); monto DECIMAL(12,2); referencia_externa VARCHAR(80); fecha_transaccion TIMESTAMP | PK: id_pago. FK: id_pedido → pedido |
| envio | id_envio INT; id_pedido INT; transportadora VARCHAR(60); numero_guia VARCHAR(60); estado VARCHAR(25); costo DECIMAL(12,2); fecha_despacho TIMESTAMP; fecha_entrega TIMESTAMP NULL | PK: id_envio. FK: id_pedido → pedido |
| promocion | id_promocion INT; codigo VARCHAR(30) UQ; tipo VARCHAR(20); valor DECIMAL(12,2); fecha_inicio DATE; fecha_fin DATE; usos_maximos INT; activa BOOLEAN | PK: id_promocion |

## 7.2. Relaciones y cardinalidades

**Tabla 13**
*Relaciones principales entre tablas y sus cardinalidades*

| Relación | Cardinalidad | Regla de negocio que representa |
|---|---|---|
| usuario — cliente / administrador | 1 : 1 | Un usuario del sistema es cliente o administrador, y hereda los datos de autenticación. |
| cliente — direccion | 1 : N | Un cliente puede registrar varias direcciones de envío. |
| cliente — pedido | 1 : N | Un cliente puede realizar varios pedidos; cada pedido pertenece a un único cliente. |
| cliente — carrito | 1 : N | Un cliente tiene un carrito activo a la vez, además de los carritos históricos. |
| carrito — item_carrito | 1 : N | Un carrito agrupa varios ítems; los ítems no existen sin el carrito. |
| pedido — detalle_pedido | 1 : N | Un pedido contiene al menos una línea de detalle. |
| pedido — pago | 1 : N | Un pedido puede registrar varios intentos de pago, aunque solo uno se confirma. |
| pedido — envio | 1 : 1 | Cada pedido despachado genera un único registro de envío. |
| promocion — pedido | 1 : N | Un cupón puede aplicarse a varios pedidos; cada pedido aplica a lo sumo un cupón. |
| categoria — categoria | 1 : N | Una categoría puede contener subcategorías (relación reflexiva). |
| categoria — producto | 1 : N | Una categoría agrupa varios productos; cada producto pertenece a una categoría. |
| producto — material | N : M | Una pieza puede combinar varios materiales y un material está presente en varias piezas; se resuelve con producto_material. |
| producto — variante_producto | 1 : N | Un producto se comercializa en una o varias variantes. |
| variante_producto — item_carrito / detalle_pedido | 1 : N | La variante es la unidad que se agrega al carrito y se vende en un pedido. |
| variante_producto — movimiento_inventario | 1 : N | Cada variante acumula el historial de movimientos que explica su existencia actual. |
| producto / categoria — metadato_seo | 1 : 1 | Cada producto o categoría publicada tiene un conjunto de metadatos asociado. |
| producto — resena | 1 : N | Un producto puede recibir varias reseñas de clientes. |

## 7.3. Consideraciones de integridad y normalización

El modelo se encuentra en tercera forma normal: no existen grupos repetitivos, los atributos dependen por completo de la clave primaria de su tabla y no se presentan dependencias transitivas entre atributos no clave (Date, 2001). Las tablas producto_material e item_carrito eliminan las relaciones muchos a muchos, y los atributos derivados —como el total del pedido— se calculan a partir del detalle, aunque se almacenan en pedido para conservar el valor histórico de la transacción.

La integridad referencial se garantiza mediante claves foráneas con restricciones de eliminación diferenciadas: las líneas de detalle se eliminan con su pedido o su carrito (composición), mientras que un producto con pedidos asociados no puede eliminarse, sino que cambia su estado de publicación, de modo que el histórico de ventas permanezca consistente. Adicionalmente, el precio unitario se replica en detalle_pedido para que las modificaciones posteriores del precio del catálogo no alteren los pedidos ya registrados.

## 8. Integración de los modelos

Los elementos desarrollados en este documento no son piezas independientes: cada uno se deriva del anterior y encuentra su correspondencia en el siguiente. La cadena de trazabilidad parte de los problemas de negocio identificados en Nalu, los traduce en objetivos, estos en funcionalidades concretas representadas como casos de uso, y estas en las clases y tablas que las soportan, todo ello desplegado sobre los componentes de la arquitectura definida en el apartado 4.

**Tabla 14**
*Matriz de trazabilidad entre los elementos de la documentación*

| Problema de negocio | Objetivo | Casos de uso | Clases | Tablas |
|---|---|---|---|---|
| Dependencia del contacto manual para vender | OE-2 | CU-01, CU-02, CU-03, CU-04, CU-05, CU-07 | Producto, VarianteProducto, Carrito, ItemCarrito, Pedido, Pago, Cliente | producto, variante_producto, carrito, item_carrito, pedido, pago, cliente |

| Problema de negocio | Objetivo | Casos de uso | Clases | Tablas |
|---|---|---|---|---|
| Sin registro estructurado de existencias | OE-3 | CU-06, CU-12 | VarianteProducto, MovimientoInventario | variante_producto, movimiento_inventario |
| Catálogo multimaterial sin estructura | OE-1 | CU-01, CU-11 | Producto, Material, ProductoMaterial, VarianteProducto, Categoria, ImagenProducto | producto, material, producto_material, variante_producto, categoria, imagen_producto |
| Sin trazabilidad del pedido ni de la entrega | OE-2 | CU-09, CU-13, CU-08 | Pedido, DetallePedido, Envio, Direccion | pedido, detalle_pedido, envio, direccion |
| Decisiones comerciales sin información | OE-5 | CU-16 | Pedido, DetallePedido, VarianteProducto, MovimientoInventario | pedido, detalle_pedido, variante_producto, movimiento_inventario |
| Baja visibilidad de la oferta | OE-5 | CU-15, CU-02 | MetadatoSEO, Producto, Categoria, Resena | metadato_seo, producto, categoria, resena |
| Ausencia de una estructura técnica que soporte lo anterior | OE-4 | Todos | Todas | Todas |

La lectura de la matriz permite verificar la coherencia del diseño en ambos sentidos. En sentido descendente, cada problema identificado tiene al menos un caso de uso que lo atiende y una estructura de datos que lo soporta. En sentido ascendente, ninguna clase o tabla del modelo existe sin una funcionalidad que la justifique: por ejemplo, la tabla movimiento_inventario no se incorpora por completitud técnica, sino porque CU-12 exige conservar la trazabilidad de cada cambio de existencias, requisito derivado del segundo problema de negocio.

La arquitectura, por su parte, agrupa estas funcionalidades en módulos que reproducen la misma división: el módulo de catálogo opera sobre las clases de la vista de catálogo, el módulo de inventario sobre VarianteProducto y MovimientoInventario, y el módulo de pedidos sobre Pedido, DetallePedido, Pago y Envio. De esta forma, la correspondencia entre la vista funcional, la vista estructural y la vista de despliegue del sistema se mantiene en todos los niveles del documento.

## 9. Requerimientos funcionales

Los requerimientos funcionales describen los servicios que el sistema debe proporcionar, la forma en que debe reaccionar ante entradas particulares y su comportamiento en situaciones específicas (Sommerville, 2011). En el caso de Nalu, se derivan directamente de los problemas de negocio, de los objetivos específicos y de los casos de uso definidos en los apartados anteriores, de modo que cada requerimiento corresponde a una funcionalidad real que se proyecta desarrollar dentro del producto mínimo viable descrito en el alcance de la solución.

Los requerimientos se identifican con el prefijo RF seguido de un número consecutivo. Para cada uno se indica su nombre, la descripción de la funcionalidad, el actor que interviene, el resultado esperado y los casos de uso con los que se relaciona, lo que permite mantener la trazabilidad con los modelos UML del documento (Pressman & Maxim, 2021). La Tabla 15 presenta los quince requerimientos funcionales del sistema transaccional.

**Tabla 15**
*Requerimientos funcionales del sistema transaccional Nalu*

| Código | Nombre | Descripción | Actor | Resultado esperado | CU |
|---|---|---|---|---|---|
| RF01 | Registrar y autenticar usuarios | El sistema debe permitir que un visitante cree una cuenta con nombre, apellido, correo electrónico, teléfono y contraseña, aceptando la política de tratamiento de datos, y que cualquier usuario registrado inicie y cierre sesión con su correo y contraseña. El correo debe ser único en el sistema. | Visitante, cliente registrado, administrador | La cuenta queda registrada en las tablas usuario y cliente, y el usuario accede a las funciones que corresponden a su rol. | CU-04 |
| RF02 | Gestionar perfil y direcciones del cliente | El sistema debe permitir que el cliente consulte y actualice sus datos de contacto y que registre, edite o elimine varias direcciones de envío, identificando cada una con una etiqueta (casa, oficina, etc.). | Cliente registrado | El perfil y las direcciones del cliente quedan actualizados y disponibles para seleccionarlos durante la compra. | CU-04, CU-05 |

| Código | Nombre | Descripción | Actor | Resultado esperado | CU |
|---|---|---|---|---|---|
| RF03 | Gestionar categorías del catálogo | El sistema debe permitir crear, editar, activar o desactivar categorías y subcategorías (por ejemplo, Anillos > Anillos de plata), con su nombre, su descripción y su URL amigable (slug). | Administrador | El catálogo queda organizado en una estructura jerárquica que se refleja en la navegación de la tienda. | CU-11 |
| RF04 | Gestionar productos, materiales y variantes | El sistema debe permitir registrar y actualizar cada pieza con su nombre, descripción, categoría, precio base, imágenes y estado de publicación; asociarle uno o varios materiales (acero, plata, oro, esmeralda, cuarzo) con su rol, ley, peso o quilates; y definir sus variantes comercializables con SKU único, talla, precio adicional y existencia inicial. | Administrador | El producto queda registrado con sus materiales y variantes, y se publica en la tienda cuando su estado es activo. | CU-11 |
| RF05 | Gestionar contenido y metadatos | El sistema debe permitir editar, para cada producto y categoría, el título de la página, la meta descripción, las palabras clave, el texto alternativo de las imágenes, la URL canónica y los datos estructurados basados en el vocabulario Schema.org (Product y Offer). | Administrador | Las páginas públicas presentan contenido descriptivo y metadatos completos que pueden ser interpretados por motores de búsqueda y por sistemas de IA. | CU-15 |
| RF06 | Consultar y filtrar el catálogo | El sistema debe mostrar el listado de productos publicados con su imagen principal, nombre, material y precio de venta, y permitir buscarlos por texto y filtrarlos por categoría, material, rango de precio y disponibilidad. | Visitante, cliente registrado | El usuario obtiene un listado actualizado de piezas que cumplen los criterios, consistente con el inventario registrado. | CU-01 |
| RF07 | Consultar ficha de producto y registrar reseñas | El sistema debe presentar la ficha detallada de cada pieza: imágenes, descripción, materiales, ley del metal, peso, piedras, tallas disponibles, precio y reseñas aprobadas. El cliente que haya recibido un pedido con el producto podrá registrar una calificación de 1 a 5 y un comentario. | Visitante, cliente registrado, administrador (aprobación de reseñas) | El comprador dispone de la información necesaria para decidir la compra sin contactar al propietario, y las reseñas quedan registradas para su moderación. | CU-02 |
| RF08 | Gestionar el carrito de compras | El sistema debe permitir agregar variantes al carrito, modificar cantidades y eliminar ítems, recalculando el subtotal con el precio vigente. El carrito del visitante se conserva mediante un identificador de sesión y se asocia a la cuenta al iniciar sesión. | Visitante, cliente registrado | El carrito queda registrado con sus ítems, cantidades y subtotal actualizado. | CU-03 |

| Código | Nombre | Descripción | Actor | Resultado esperado | CU |
|---|---|---|---|---|---|
| RF09 | Validar y reservar existencias | Antes de confirmar un pedido, el sistema debe verificar la disponibilidad real de cada variante (existencia actual menos unidades reservadas) y reservar las unidades durante el proceso de pago; si el pago se rechaza o expira, debe liberar la reserva automáticamente. | Sistema (invocado por el cliente registrado) | Ninguna pieza se vende sin existencia y la disponibilidad publicada coincide con la disponibilidad real. | CU-06 |
| RF10 | Registrar el pedido | El sistema debe convertir el carrito en un pedido con número único, dirección de envío, detalle de piezas con su precio unitario histórico, subtotal, descuento, costo de envío y total, dejándolo en estado pendiente de pago y redirigiendo al cliente a la pasarela de pagos para completar el cobro. | Cliente registrado, pasarela de pagos | Queda registrado un pedido completo y trazable, con la referencia que lo vincula a la transacción en la pasarela de pagos. | CU-05 |
| RF11 | Gestionar promociones y aplicar cupones | El sistema debe permitir al administrador crear cupones con código, tipo (porcentaje o valor fijo), valor, vigencia y número máximo de usos, y permitir al cliente aplicar un cupón válido durante la compra. | Administrador, cliente registrado | El descuento se calcula y se registra en el pedido solo cuando el cupón está vigente y tiene usos disponibles. | CU-10, CU-14 |
| RF12 | Procesar pagos mediante la pasarela de pagos | El sistema debe integrarse con una pasarela de pagos externa (por ejemplo, Wompi) que permita pagar con tarjeta débito o crédito, PSE y billeteras digitales. Debe enviar a la pasarela el monto y la referencia del pedido, recibir la confirmación mediante una notificación asincrónica (webhook) con firma de integridad y registrar el método, el estado (aprobado, rechazado, pendiente o anulado), el monto y la referencia externa de cada intento de pago. Los datos de la tarjeta se capturan solo en la pasarela. | Cliente registrado, pasarela de pagos | El pedido cambia a estado pagado únicamente cuando la pasarela confirma la transacción; si el pago es rechazado o expira, el pedido conserva su estado, se libera la reserva y el cliente puede reintentar el pago. | CU-07 |
| RF13 | Registrar movimientos de inventario | El sistema debe permitir registrar entradas, salidas y ajustes de existencias por variante, indicando cantidad y motivo, y registrar automáticamente las salidas generadas por los pedidos pagados y los reintegros por cancelación. | Administrador, sistema | La existencia de cada variante queda actualizada y cada cambio conserva su fecha, su tipo y el usuario responsable. | CU-12 |

| Código | Nombre | Descripción | Actor | Resultado esperado | CU |
|---|---|---|---|---|---|
| RF14 | Gestionar pedidos, envíos y seguimiento | El sistema debe permitir al administrador consultar los pedidos por estado o fecha, cambiar su estado (pagado, en preparación, enviado, entregado, cancelado), registrar la transportadora y el número de guía, y notificar al cliente cada cambio. El cliente debe poder consultar el estado y el historial de sus pedidos. | Administrador, cliente registrado, operador logístico | Cada pedido refleja su estado real y el cliente cuenta con la información de seguimiento sin necesidad de preguntar por ella. | CU-08, CU-09, CU-13 |
| RF15 | Generar reportes de ventas e inventario | El sistema debe generar reportes de ventas por periodo, material y categoría, de productos con mayor rotación, de variantes agotadas o con existencia baja, de uso de cupones y de pagos por estado y método, con su referencia en la pasarela de pagos para facilitar la conciliación, con opción de exportarlos en formato CSV. | Administrador | El propietario dispone de información verificable para tomar decisiones de reposición, precio y promoción. | CU-16 |

*Nota.* CU = casos de uso relacionados, definidos en el apartado 5. Elaboración propia.

Los requerimientos RF01 a RF05 conforman la base de información del sistema: sin usuarios, categorías, productos y contenido no existe catálogo que publicar. Los requerimientos RF06 a RF12 cubren el ciclo transaccional de la venta, desde la consulta del catálogo hasta la confirmación del pago, y son los que permiten que el emprendimiento venda sin depender del contacto manual. Dentro de este grupo, RF10 y RF12 definen la integración con la pasarela de pagos: el sistema no cobra directamente, sino que delega la autorización del pago en un servicio externo certificado y registra su resultado para cada pedido. Por último, RF13 a RF15 dan soporte a la administración del negocio: mantienen la consistencia del inventario, permiten seguir cada pedido hasta su entrega y convierten las operaciones registradas en información para la toma de decisiones.

## 10. Requerimientos no funcionales

Los requerimientos no funcionales establecen las restricciones y las propiedades de calidad que el sistema debe cumplir, como la seguridad, el rendimiento, la usabilidad o la disponibilidad, y suelen afectar al sistema en su conjunto más que a una funcionalidad aislada (Sommerville, 2011). Para clasificarlos se toma como referencia el modelo de calidad del producto de la norma ISO/IEC 25010:2023, que organiza la calidad del software en características como la adecuación funcional, la eficiencia de desempeño, la compatibilidad, la capacidad de interacción, la fiabilidad, la seguridad, la mantenibilidad y la flexibilidad (International Organization for Standardization [ISO], 2023).

Cada requerimiento se redacta con un criterio verificable, de manera que su cumplimiento pueda comprobarse mediante pruebas, inspecciones o mediciones durante la construcción del prototipo. Los criterios de seguridad se apoyan en el estándar de verificación de seguridad de aplicaciones de OWASP (OWASP Foundation, 2025); los de rendimiento, en las métricas Core Web Vitals (Google, s.f.-a); los de accesibilidad, en las pautas WCAG 2.2 (World Wide Web Consortium [W3C], 2023); y los de protección de datos, en la Ley 1581 de 2012. La Tabla 16 presenta los quince requerimientos no funcionales.

**Tabla 16**
*Requerimientos no funcionales del sistema transaccional Nalu*

| Código | Característica de calidad | Requerimiento | Criterio de verificación |
|---|---|---|---|
| RNF01 | Seguridad (autenticidad) | Las contraseñas deben almacenarse únicamente como resumen criptográfico con un algoritmo adaptativo (bcrypt o Argon2) y exigir una longitud mínima de 8 caracteres. La sesión debe expirar tras 30 minutos de inactividad. | Inspección de la tabla usuario: ningún registro contiene contraseñas en texto plano. Prueba: una sesión inactiva por 30 minutos exige autenticarse de nuevo. |

| Código | Característica de calidad | Requerimiento | Criterio de verificación |
|---|---|---|---|
| RNF02 | Seguridad (control de acceso) | Las funciones del panel administrativo y sus servicios deben estar disponibles solo para usuarios con rol de administrador; un cliente únicamente puede consultar sus propios pedidos, direcciones y datos. | Pruebas sobre el 100 % de las rutas administrativas sin sesión o con rol cliente: todas responden con código 401 o 403. Un cliente que solicita un pedido ajeno recibe 403 o 404. |
| RNF03 | Seguridad (confidencialidad en tránsito) | Toda comunicación entre el navegador, el servidor y los servicios externos debe realizarse por HTTPS con TLS 1.2 o superior. Las notificaciones de la pasarela de pagos solo se aceptan si su firma de integridad es válida. | Las solicitudes HTTP se redirigen a HTTPS (código 301). Un webhook con firma alterada es rechazado y no modifica el estado del pago. |
| RNF04 | Seguridad (datos de pago) | El sistema no debe recibir, procesar ni almacenar números de tarjeta, códigos de seguridad ni credenciales bancarias; estos datos se capturan exclusivamente en la pasarela de pagos, que cumple el estándar PCI DSS (PCI Security Standards Council, 2024). | Revisión del modelo de datos y de los registros de la aplicación: la tabla pago solo almacena método, estado, monto y referencia externa. |
| RNF05 | Protección de datos personales | El tratamiento de los datos de los clientes debe cumplir la Ley 1581 de 2012: autorización expresa al registrarse, política de tratamiento visible y mecanismos para consultar, actualizar y solicitar la supresión de los datos. | El formulario de registro no permite continuar sin aceptar la política. Existe la opción de actualizar datos y solicitar supresión desde el perfil. |
| RNF06 | Eficiencia de desempeño (tiempo de respuesta) | Las páginas de catálogo y de ficha de producto deben cargar su contenido principal en 2,5 segundos o menos, y las operaciones de carrito y confirmación de pedido deben responder en 2 segundos o menos, sin incluir el tiempo de la pasarela. | Medición con Lighthouse o PageSpeed Insights: LCP ≤ 2,5 s en el percentil 75. Prueba de carga con 50 usuarios concurrentes: el 95 % de las respuestas de la API en ≤ 2 s. |
| RNF07 | Interoperabilidad semántica (visibilidad) | Las páginas de producto y categoría deben generarse desde el servidor (renderizado en el servidor de Angular) e incluir título, meta descripción, URL canónica y datos estructurados Schema.org Product en formato JSON-LD, para que puedan ser interpretadas por motores de búsqueda y sistemas de IA. | El 100 % de las fichas publicadas supera sin errores la validación de datos estructurados (Schema Markup Validator o Prueba de resultados enriquecidos de Google). |
| RNF08 | Capacidad de interacción (usabilidad) | El proceso de compra debe completarse en un máximo de cinco pasos desde la ficha de producto hasta la confirmación del pago, con mensajes de error en español que indiquen cómo corregir el dato. | Prueba de usabilidad con cinco usuarios: al menos cuatro completan una compra de prueba sin ayuda en menos de 5 minutos. |

| Código | Característica de calidad | Requerimiento | Criterio de verificación |
|---|---|---|---|
| RNF09 | Capacidad de interacción (accesibilidad) | La tienda debe cumplir los criterios de nivel AA de WCAG 2.2 relacionados con contraste de color, navegación por teclado, etiquetas de formulario y texto alternativo de las imágenes. | El 100 % de las imágenes de producto tiene texto alternativo. Auditoría automática (Lighthouse/axe) sin errores críticos de accesibilidad. |
| RNF10 | Integridad de los datos (transacciones) | Las operaciones que afectan varias tablas —crear el pedido, registrar su detalle, reservar o descontar inventario y registrar el pago— deben ejecutarse como una única transacción ACID; las existencias nunca pueden ser negativas. | Prueba con fallo simulado a mitad del proceso: no queda un pedido sin detalle ni inventario descontado. Restricción CHECK (stock_actual >= 0) definida en la base de datos. |
| RNF11 | Integridad de los datos (concurrencia y validación) | Dos compras simultáneas de la última unidad de una variante no pueden confirmarse ambas. Los datos de entrada deben validarse en el servidor: precios mayores que cero, SKU y correo únicos, cantidades enteras positivas. | Prueba concurrente de dos pedidos sobre una variante con existencia 1: solo uno se confirma. Envío de datos inválidos a la API: el sistema responde con error 400 y no persiste el registro. |
| RNF12 | Fiabilidad (disponibilidad y recuperación) | La tienda debe estar disponible al menos el 99 % del tiempo mensual. La base de datos debe respaldarse automáticamente cada 24 horas, con retención mínima de 7 días. | Monitoreo de disponibilidad mensual ≥ 99 % (máximo ≈ 7,3 h fuera de servicio). Restauración de una copia de respaldo en un entorno de prueba en menos de 2 horas. |
| RNF13 | Mantenibilidad | El código debe organizarse en los módulos definidos en la arquitectura, gestionarse con control de versiones (Git) y contar con pruebas automatizadas para las reglas de negocio de pedidos, inventario y promociones. | Cobertura de pruebas unitarias (JUnit) ≥ 60 % en los módulos de pedidos, inventario y promociones. Un cambio en el módulo de promociones no exige modificar otros módulos. |
| RNF14 | Compatibilidad | La tienda y el panel administrativo deben funcionar en las dos versiones más recientes de Chrome, Firefox, Safari y Edge, y adaptarse a pantallas desde 360 px (móvil) hasta 1920 px (escritorio). | Lista de verificación ejecutada en cada navegador y en tres anchos de pantalla (360, 768 y 1366 px) sin pérdida de funciones ni desplazamiento horizontal. |
| RNF15 | Trazabilidad | El sistema debe registrar quién, cuándo y qué operación realizó en los movimientos de inventario, los cambios de estado de los pedidos, los cambios de precio y las acciones del panel administrativo. | Para cualquier cambio de existencia o de estado de un pedido es posible identificar la fecha, el usuario responsable y el motivo, con una retención mínima de un año. |

*Nota.* Las características de calidad se nombran según el modelo de la norma ISO/IEC 25010:2023. Elaboración propia.

Los requerimientos no funcionales complementan a los funcionales y condicionan las decisiones técnicas ya documentadas. RNF10 y RNF11 justifican la elección de una base de datos relacional con transacciones ACID descrita en la arquitectura; RNF07 explica por qué las

páginas de catálogo se generan desde el servidor; y RNF15 se materializa en la tabla movimiento_inventario del modelo relacional. De este modo, las condiciones de calidad no se añaden al final del desarrollo, sino que forman parte del diseño desde su inicio.

# 11. Marco conceptual

El marco conceptual reúne los conceptos que fundamentan el diseño del sistema transaccional de Nalu. Los conceptos se agrupan en cuatro bloques: los propios del negocio y de la solución propuesta, los de los sistemas transaccionales, los de las tecnologías previstas y, finalmente, su aplicación concreta en el prototipo.

## 11.1. Conceptos fundamentales del proyecto

### *Comercio electrónico*

El comercio electrónico se define como el uso de internet, la web y las aplicaciones móviles para realizar transacciones comerciales entre organizaciones e individuos, entendiendo por transacción comercial el intercambio de valor a cambio de productos o servicios (Laudon & Traver, 2021). Para Nalu, esto significa pasar de un modelo de venta por pedido, en el que la transacción solo comienza cuando el cliente escribe al propietario, a un modelo de tienda en línea en el que el catálogo, el precio, la disponibilidad y el pago están disponibles de forma permanente y el cliente puede completar la compra por sí mismo. La modalidad que corresponde al proyecto es la de comercio electrónico de empresa a consumidor (B2C).

### *Catálogo, producto y variante*

En un catálogo de joyería conviene distinguir entre el producto, que describe la pieza de forma general (por ejemplo, un anillo solitario con esmeralda), y la variante, que es la unidad que efectivamente se vende y se cuenta en inventario (el mismo anillo en talla 7, con su propio código SKU y su propia existencia). Esta distinción, presente en el diagrama de clases mediante las clases Producto y VarianteProducto, permite que el cliente vea una sola ficha por pieza mientras el sistema controla la disponibilidad de cada talla de forma independiente.

### *Atributos de materiales en joyería*

La ampliación del catálogo introduce atributos que no existían cuando el negocio vendía solo plata. La ley de un metal indica la proporción de metal precioso en la aleación: la plata de ley 925 contiene 92,5 % de plata, y el oro de 18 quilates contiene 75 % de oro. Las piedras, como la esmeralda o el cuarzo, se describen por su peso en quilates. Estos atributos influyen en el precio y en la decisión de compra, por lo que el sistema los registra en las entidades Material y ProductoMaterial y los presenta en la ficha del producto.

### *Gestión de inventario*

La gestión de inventario consiste en controlar las existencias disponibles para la venta mediante el registro de cada entrada, salida y ajuste. En lugar de modificar directamente un número de existencias, el sistema registra movimientos que explican cada cambio, de modo que la existencia actual puede reconstruirse y auditarse. A este control se suma la reserva de existencias: las unidades de un pedido pendiente de pago se apartan temporalmente para que no se vendan dos veces.

### *Optimización para motores de búsqueda y motores generativos*

La optimización para motores de búsqueda (SEO) agrupa las prácticas que facilitan que una página sea rastreada, comprendida y presentada en los resultados de búsqueda (Enge et al., 2015). Con la aparición de los motores de búsqueda basados en modelos de lenguaje, que responden con texto generado a partir de varias fuentes, surge la optimización para motores generativos (GEO), definida como el conjunto de estrategias orientadas a aumentar la visibilidad de un contenido dentro de esas respuestas generadas (Aggarwal et al., 2024). Ambas prácticas se apoyan en contenido descriptivo de calidad y en datos estructurados, es decir, en información marcada con un vocabulario común como Schema.org, que permite indicar de forma explícita

que una página describe un producto con un nombre, un precio, una disponibilidad y una marca (Schema.org, 2024).

## 11.2. Conceptos de sistemas transaccionales

### *Transacción y propiedades ACID*

Una transacción es una unidad lógica de procesamiento de la base de datos que incluye una o más operaciones de lectura o escritura y que debe ejecutarse por completo o no ejecutarse en absoluto (Elmasri & Navathe, 2016). Las transacciones deben cumplir las propiedades ACID: atomicidad (todas las operaciones se confirman o ninguna), consistencia (la base de datos pasa de un estado válido a otro estado válido), aislamiento (las transacciones concurrentes no interfieren entre sí) y durabilidad (los cambios confirmados no se pierden ante un fallo). En Nalu, confirmar un pedido implica crear el pedido, registrar su detalle, reservar o descontar el inventario y registrar el pago; si alguna de estas operaciones falla, todas deben revertirse.

### *Procesamiento de transacciones en línea*

Los sistemas de procesamiento de transacciones en línea (OLTP) se caracterizan por ejecutar un gran número de operaciones cortas —registros, consultas y actualizaciones— por parte de muchos usuarios concurrentes, con prioridad en la integridad y en el tiempo de respuesta (Elmasri & Navathe, 2016). Las operaciones básicas sobre los datos se resumen en crear, consultar, actualizar y eliminar registros (CRUD). En el sistema de Nalu, registrar un cliente o un producto corresponde a una creación; filtrar el catálogo, a una consulta; cambiar el estado de un pedido o la existencia de una variante, a una actualización; y retirar un ítem del carrito, a una eliminación. Los reportes de ventas, en cambio, son operaciones de consulta y agregación sobre los datos ya registrados.

### *Control de concurrencia*

Cuando varios usuarios intentan modificar el mismo dato al mismo tiempo, el sistema de gestión de bases de datos debe evitar que una actualización sobrescriba a otra o que se lean datos inconsistentes. PostgreSQL resuelve esta situación con el control de concurrencia multiversión (MVCC) y con bloqueos explícitos a nivel de fila, como la instrucción SELECT … FOR UPDATE, que impide que otra transacción modifique la fila hasta que la primera termine (PostgreSQL Global Development Group, s.f.). En Nalu, este mecanismo evita que dos clientes compren al mismo tiempo la última unidad de una pieza.

### *Integridad referencial y estados de la transacción*

La integridad referencial garantiza que toda clave foránea apunte a un registro existente, de modo que no puedan existir, por ejemplo, detalles de pedido sin pedido o pagos asociados a pedidos inexistentes (Date, 2001). Además, en un sistema transaccional las entidades de negocio pasan por estados definidos: un pedido avanza de pendiente a pagado, en preparación, enviado y entregado, o puede cancelarse. Restringir las transiciones permitidas entre estos estados evita, por ejemplo, despachar un pedido cuyo pago no fue confirmado.

### *Notificaciones asincrónicas e idempotencia*

Algunas operaciones dependen de sistemas externos que responden en un momento posterior. Las pasarelas de pago informan el resultado de una transacción mediante eventos o webhooks, es decir, solicitudes HTTP que la pasarela envía al servidor del comercio cuando cambia el estado del pago (Wompi, s.f.). Como una misma notificación puede llegar más de una vez, su procesamiento debe ser idempotente: recibir el mismo evento dos veces no debe registrar dos pagos ni descontar dos veces el inventario.

### 11.3. Conceptos tecnológicos

#### *Arquitectura cliente-servidor, HTTP y API REST*

En la arquitectura cliente-servidor, el navegador del usuario solicita recursos y el servidor los procesa y responde mediante el protocolo HTTP, en su versión cifrada HTTPS. Sobre esta base, el estilo arquitectónico REST organiza la comunicación en torno a recursos identificados por URL y manipulados mediante operaciones estándar del protocolo, con interacciones sin estado entre cliente y servidor (Fielding, 2000). Los mensajes se intercambian en formato JSON, que es el que utiliza la interfaz entre la capa de presentación y la capa de lógica de negocio descrita en el apartado 4.3.

#### *Spring Boot y Java*

La capa de lógica de negocio se construirá con Spring Boot, un marco de trabajo basado en Java que permite crear aplicaciones Spring autónomas y listas para producción, con configuración automática y un servidor web embebido (Spring, s.f.-b). Spring Boot facilita exponer la API REST que consume la capa de presentación mediante controladores que reciben y devuelven JSON, y organizar el código en los módulos definidos en el apartado 4 —catálogo, inventario, pedidos, clientes, promociones, contenido y reportes— como paquetes con servicios y repositorios independientes. Para la persistencia se utilizará Spring Data JPA, que traduce las entidades del diagrama de clases en tablas del modelo relacional, y para la seguridad, Spring Security, que gestiona la autenticación, el cifrado de contraseñas y el control de acceso por rol exigidos por RNF01 y RNF02. Un aspecto central para el proyecto es la gestión declarativa de transacciones de Spring: al marcar un método de servicio como transaccional, todas las operaciones que ejecuta sobre la base de datos se confirman o se revierten en conjunto (Spring,

s.f.-a), lo que permite implementar directamente la confirmación atómica del pedido descrita en RNF10.

***Angular y renderizado en el servidor***

La capa de presentación —tienda en línea y panel administrativo— se construirá con Angular, un marco de trabajo de código abierto para aplicaciones web basado en TypeScript y organizado en componentes reutilizables (Google, s.f.-b). Angular consume la API REST de Spring Boot mediante su cliente HTTP y permite construir formularios con validación, rutas protegidas por rol y vistas adaptables a distintos tamaños de pantalla. Dado que una aplicación Angular se ejecuta por defecto en el navegador, para las páginas públicas del catálogo se utilizará el renderizado en el servidor (SSR) de Angular, que construye el HTML completo de cada página en el servidor, de modo que el contenido se muestra antes y los motores de búsqueda pueden indexarlo con mayor facilidad (Google, s.f.-c). Esta decisión responde a RNF06 y RNF07 y a la necesidad de visibilidad del catálogo frente a buscadores y sistemas de IA.

***PostgreSQL***

Como base de datos se utilizará PostgreSQL, un sistema de gestión de bases de datos relacional de código abierto que implementa transacciones ACID, claves foráneas, restricciones de verificación (CHECK) y tipos de datos como JSON (PostgreSQL Global Development Group, s.f.). Estas capacidades permiten implementar directamente el modelo relacional del apartado 7: las claves foráneas aseguran la integridad referencial, las restricciones impiden existencias negativas y el tipo JSON almacena los datos estructurados de la tabla metadato_seo. La aplicación Spring Boot se conecta a PostgreSQL mediante JDBC y JPA, de modo que la base de datos relacional es la única fuente de verdad de todas las transacciones del sistema.

***Pasarela de pagos***

Una pasarela de pagos es un servicio que autoriza y procesa pagos electrónicos —tarjetas, transferencias o billeteras— en nombre del comercio, de manera que este no maneja directamente los datos financieros del comprador. En el contexto colombiano existen pasarelas como Wompi, que exponen una API para crear transacciones y notifican su resultado mediante eventos firmados (Wompi, s.f.). Delegar el cobro en una pasarela certificada reduce el riesgo del emprendimiento y lo libera de las exigencias de la norma PCI DSS sobre almacenamiento de datos de tarjeta (PCI Security Standards Council, 2024).

***Modelos de lenguaje e inteligencia artificial generativa***

Los modelos de lenguaje de gran tamaño son modelos de inteligencia artificial entrenados con grandes volúmenes de texto que pueden comprender instrucciones y generar texto en lenguaje natural. Su principal limitación es que pueden producir respuestas plausibles pero incorrectas cuando no disponen de la información necesaria. La generación aumentada por recuperación (RAG) mitiga este problema al proporcionar al modelo, en cada consulta, información recuperada de una fuente confiable para que fundamente su respuesta (Lewis et al., 2020). De forma similar, la llamada a funciones permite que el modelo solicite a la aplicación ejecutar una operación definida —por ejemplo, consultar el catálogo— y utilice el resultado real para responder (Google Cloud, s.f.). Estos conceptos sustentan el servicio de IA propuesto en el apartado 13.

### 11.4. Aplicación de los conceptos al proyecto

La Tabla 17 resume cómo se materializa cada concepto en el producto que se está diseñando y con qué requerimientos se relaciona, de modo que el marco conceptual no quede

como un conjunto de definiciones aisladas, sino como el fundamento de decisiones concretas del sistema.

**Tabla 17**

*Aplicación de los conceptos al sistema transaccional Nalu*

| Concepto | Aplicación en el prototipo | Requerimientos |
|---|---|---|
| Comercio electrónico B2C | Tienda en línea disponible de forma permanente, en la que el cliente consulta, compra y paga sin contactar al propietario. | RF06–RF12 |
| Producto y variante | Una ficha por pieza con varias variantes (talla, peso), cada una con SKU y existencia propios. | RF04, RF08, RF09 |
| Atributos de materiales | Registro de ley, peso y quilates por material, mostrados en la ficha y usados en los filtros. | RF04, RF06, RF07 |
| Gestión de inventario por movimientos | Tabla movimiento_inventario y reserva de unidades durante el pago. | RF09, RF13, RNF15 |
| SEO, GEO y datos estructurados | Metadatos y JSON-LD Schema.org por producto y categoría; páginas generadas en el servidor. | RF05, RNF07 |
| Transacción ACID | Creación del pedido, detalle, inventario y pago en una sola transacción. | RF10, RF12, RNF10 |
| Control de concurrencia | Bloqueo de la fila de la variante al reservar existencias. | RF09, RNF11 |
| Estados e integridad referencial | Transiciones de estado controladas del pedido; claves foráneas del modelo relacional. | RF10, RF14 |
| Webhooks e idempotencia | Confirmación del pago por evento firmado, procesado una sola vez. | RF12, RNF03 |
| API REST y JSON | Comunicación entre la tienda, el panel y los módulos de negocio. | Todos los RF |
| Spring Boot, Angular y PostgreSQL | Spring Boot para la API REST y la lógica de negocio con transacciones declarativas; Angular con SSR para la tienda y el panel; PostgreSQL como base de datos relacional. | RNF01, RNF02, RNF06, RNF10, RNF13, RNF14 |
| Modelos de lenguaje, RAG y llamada a funciones | Asistente de compra y asistente de contenido fundamentados en datos reales del catálogo. | Apartado 13 |

*Nota.* Elaboración propia.

# 12. Marco contextual

## 12.1. Contexto del proyecto

Nalu es un emprendimiento colombiano de joyería, de tamaño pequeño, administrado directamente por su propietario. Hasta ahora ha comercializado piezas en plata bajo un modelo de venta por pedido, en el que la relación con el cliente ocurre por mensajería o en persona. El negocio se encuentra en un momento de transición: ha decidido ampliar su oferta a piezas en acero, oro, esmeraldas y cuarzo, al tiempo que sus ventas han disminuido por depender de que los clientes inicien el contacto.

Este proceso ocurre en un mercado que ya compra en línea. Durante 2025 se registraron en Colombia 684,6 millones de transacciones de comercio electrónico por un valor de 145,4 billones de pesos (Cámara Colombiana de Comercio Electrónico [CCCE], 2026), lo que indica que el comprador potencial de Nalu está habituado a buscar, comparar y pagar en línea. A la vez, la venta en línea en Colombia se desarrolla dentro de un marco normativo que el sistema debe respetar: el Estatuto del Consumidor (Ley 1480 de 2011) exige a quienes venden por medios electrónicos informar de manera veraz y suficiente sobre el producto, su precio total y las condiciones de la compra, y reconoce al consumidor el derecho de retracto; y la Ley 1581 de 2012 regula el tratamiento de los datos personales de los clientes.

El proyecto se desarrolla además en un contexto académico, como trabajo de la asignatura Sistemas Transaccionales del programa de Ingeniería de Software de la Universidad Manuela Beltrán, lo que permite aplicar el ciclo de análisis y diseño sobre un caso de negocio real.

### 12.2. Problemática identificada

Como se expuso en el apartado de problemática, los procesos comerciales de Nalu —publicación de la oferta, control de existencias, registro de pedidos, cobro y entrega— no están soportados por ningún sistema de información. Esto genera cuatro efectos concretos: la venta depende del contacto manual y de la disponibilidad del propietario; no existe registro estructurado de precios, existencias, pedidos ni clientes; la ampliación a un catálogo multimaterial aumenta el riesgo de ofrecer piezas sin existencia o con información incompleta; y la oferta tiene baja visibilidad fuera del círculo de contactos actual. En consecuencia, el negocio no puede vender de forma continua ni tomar decisiones con información verificable.

### 12.3. Usuarios y beneficiarios

El sistema tiene dos grupos de usuarios directos, el administrador y los compradores, y se relaciona con servicios externos que también participan en las transacciones. La Tabla 18 identifica sus necesidades y la forma en que el sistema las atiende.

**Tabla 18**
*Usuarios y beneficiarios del sistema*

| Usuario o beneficiario | Necesidades | Cómo las atiende el sistema |
|---|---|---|
| Propietario del emprendimiento (administrador) | Publicar y mantener un catálogo multimaterial sin errores; saber qué tiene en existencia; atender pedidos sin responder manualmente cada consulta; conocer qué se vende y qué no. | Panel administrativo para productos, inventario, pedidos, promociones, contenido y reportes (RF03–RF05, RF11, RF13–RF15). |
| Visitante (comprador potencial) | Encontrar piezas de su interés, conocer material, ley, talla, precio y disponibilidad en cualquier momento, sin depender de que alguien responda. | Catálogo con búsqueda y filtros, fichas completas y reseñas (RF06, RF07); asistente de compra con IA (apartado 13). |
| Cliente registrado | Comprar de forma segura, pagar por un medio confiable y saber en qué estado está su pedido. | Carrito, checkout, pago por pasarela y seguimiento del pedido (RF08–RF12, RF14). |
| Pasarela de pagos y operador logístico (servicios externos) | Recibir solicitudes claras con la referencia del pedido y notificar el resultado de su operación. | Integración por API REST y webhooks; registro de referencia externa y número de guía (RF12, RF14). |

*Nota.* Elaboración propia.

### 12.4. Necesidad que se pretende atender

La necesidad central de Nalu es contar con un canal de venta permanente y estructurado que no dependa del contacto manual. Concretamente, el negocio necesita publicar su catálogo con información completa y actualizada; controlar las existencias de cada variante para no comprometer piezas inexistentes; registrar cada pedido con su pago y su envío; y disponer de información confiable para administrar el negocio. A esta necesidad operativa se suma una oportunidad de mejora: hacer que la oferta sea encontrada por compradores que hoy no conocen el emprendimiento, tanto en buscadores tradicionales como en asistentes basados en inteligencia artificial.

### 12.5. Descripción del producto o prototipo

El producto es un sistema transaccional de comercio electrónico para la joyería Nalu, compuesto por una tienda en línea para los compradores y un panel administrativo para el propietario, que comparten una misma lógica de negocio y una misma base de datos relacional. Su propósito es estructurar y registrar los procesos de publicación de catálogo, control de inventario, pedidos, pagos y envíos, de manera que el emprendimiento pueda vender de forma continua y administrar el negocio con información trazable.

El sistema gestiona información de cinco tipos: información del catálogo (categorías, productos, materiales, variantes, imágenes y precios); información de inventario (existencias, reservas y movimientos); información de clientes (cuentas, perfiles y direcciones); información transaccional (carritos, pedidos, detalles, pagos, envíos y promociones); e información de contenido (descripciones, metadatos, datos estructurados y reseñas). Sus funcionalidades principales son las quince descritas en el apartado 9.

El uso esperado del sistema puede ilustrarse con un caso típico. Una compradora busca en internet un anillo de esmeralda en oro y encuentra la ficha de una pieza de Nalu, cuyos datos estructurados permitieron que el buscador la mostrara. En la ficha consulta el material, la ley del oro, el peso de la piedra y las tallas disponibles; elige su talla, la agrega al carrito, se registra, selecciona su dirección y aplica un cupón. El sistema reserva la unidad, la redirige a la pasarela de pagos y, cuando esta confirma el pago, cambia el pedido a estado pagado, descuenta el inventario y le envía la confirmación. Del otro lado, el propietario ve el nuevo pedido en el panel, lo prepara, registra la guía de la transportadora y el sistema notifica a la compradora, que puede consultar el estado de su envío desde su cuenta. Durante todo el proceso, nadie tuvo que responder un mensaje manualmente.

### 12.6. Alcance del proyecto

Durante esta etapa académica el proyecto comprende dos entregables. El primero es la documentación de análisis y diseño contenida en este documento: problemática, objetivos, arquitectura, casos de uso, diagrama de clases, modelo relacional, requerimientos y marcos conceptual, contextual e integrador. El segundo es un prototipo funcional del producto mínimo viable, construido con Spring Boot, Angular y PostgreSQL, que implemente el ciclo principal de la venta: registro y autenticación de usuarios, gestión de catálogo con materiales y variantes, consulta y filtrado del catálogo, carrito, registro del pedido con reserva de existencias, pago en el entorno de pruebas de la pasarela, gestión de estados del pedido y reportes básicos.

Quedan fuera del alcance de esta etapa: el procesamiento de pagos reales en producción, la integración automática con la API de un operador logístico (el número de guía se registra manualmente), la facturación electrónica, la integración con marketplaces, los programas de

fidelización y la aplicación móvil nativa. El servicio de inteligencia artificial descrito en el apartado 13 se documenta a nivel conceptual y no se implementa en esta entrega.

## 13. Marco integrador: incorporación del servicio de inteligencia artificial

El marco integrador describe el servicio de inteligencia artificial que se propone incorporar al sistema transaccional, su función dentro del proyecto, la forma en que se integraría con la arquitectura definida y el beneficio que se espera obtener. En esta entrega el servicio no se implementa ni se conecta; se documenta su funcionamiento conceptual y su relación con los problemas de negocio de Nalu.

### 13.1. Servicio de inteligencia artificial propuesto

Se propone utilizar la API de Gemini a través de Vertex AI, la plataforma de inteligencia artificial de Google Cloud. Gemini es una familia de modelos de lenguaje multimodales, capaces de procesar texto e imágenes y de generar texto en español, que en Vertex AI ofrece dos capacidades relevantes para el proyecto: la llamada a funciones, mediante la cual el modelo puede solicitar a la aplicación la ejecución de operaciones definidas, y la fundamentación (grounding), que permite basar las respuestas del modelo en datos proporcionados por la aplicación y no solo en su conocimiento general (Google Cloud, s.f.).

La elección se justifica por tres razones. Primero, la capacidad multimodal permite que el modelo analice la fotografía de una pieza junto con sus atributos registrados para proponer descripciones, lo que resulta útil en un catálogo de joyería en el que la imagen es determinante. Segundo, la llamada a funciones permite que el modelo responda con información real del catálogo y del inventario, en lugar de inventarla. Tercero, el modelo de cobro por uso se ajusta al volumen reducido de un emprendimiento pequeño. Otros servicios de modelos de lenguaje ofrecen capacidades equivalentes; por ello, la integración se diseña mediante un adaptador que aísla al sistema del proveedor elegido y permitiría reemplazarlo sin afectar a los demás módulos.

### 13.2. Función dentro del proyecto

El servicio cumpliría dos funciones principales, cada una asociada a un problema de negocio identificado en la problemática, y prestaría además un apoyo directo al cliente cuyos aspectos específicos aún se encuentran en evaluación.

La primera es un asistente de compra conversacional en la tienda en línea. El visitante podría escribir preguntas en lenguaje natural, como «¿Tienen anillos de plata 925 en talla 7 por menos de 200.000 pesos?» o «¿En qué estado está mi pedido?», y el asistente respondería con productos, precios y existencias reales consultados en el sistema, con enlaces a las fichas correspondientes. Esta función atiende el primer problema de negocio, la dependencia del contacto manual, porque resuelve fuera del horario del propietario las preguntas que hoy exigen que él responda una por una.

La segunda es un asistente de contenido en el panel administrativo. A partir de los atributos registrados de una pieza (material, ley, peso, piedras, talla) y de su fotografía, el servicio generaría un borrador de la descripción comercial, el título y la meta descripción de la página, el texto alternativo de las imágenes y los datos estructurados Schema.org. El administrador revisaría y aprobaría el borrador antes de publicarlo. Esta función atiende el tercer y el cuarto problema de negocio: reduce el riesgo de fichas incompletas en un catálogo multimaterial y mejora la visibilidad de la oferta en buscadores y en motores generativos (Aggarwal et al., 2024).

#### *Apoyo al cliente: aspectos en evaluación*

Además de responder consultas sobre productos y pedidos, se proyecta que la inteligencia artificial apoye al cliente durante su experiencia de compra. Sin embargo, los aspectos concretos en los que prestará este apoyo aún se están analizando y no forman parte de las decisiones

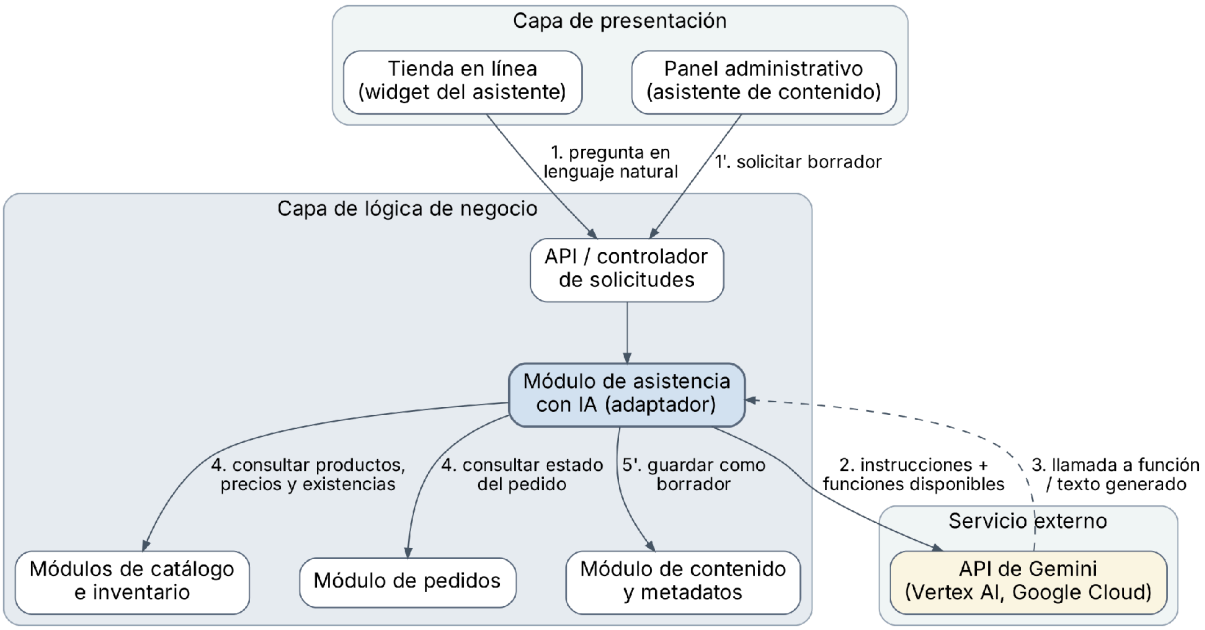
cerradas de esta entrega. Entre las posibilidades que se estudian se encuentran: la recomendación de piezas según el gusto, el presupuesto o la ocasión del cliente (por ejemplo, un regalo de aniversario); la orientación para elegir la talla de un anillo; la explicación de las diferencias entre materiales, como la ley de la plata y del oro o las características de las esmeraldas; las recomendaciones de cuidado y conservación de cada pieza; y el acompañamiento posventa, como resolver dudas sobre envíos, cambios o el derecho de retracto.

La selección de estos aspectos se realizará en las siguientes etapas del proyecto, considerando las preguntas que con mayor frecuencia reciben hoy los canales de atención de Nalu, el valor que cada función aporte al cliente, su costo de operación y los riesgos de ofrecer información incorrecta. En cualquier caso, las funciones que se incorporen seguirán el mismo principio del asistente de compra: responder con información registrada en el sistema y no con datos inventados.

### 13.3. Integración con el sistema

El servicio se incorporaría como un nuevo componente de la capa de lógica de negocio, denominado módulo de asistencia con IA, que actúa como adaptador entre el sistema y la API externa. Este módulo no accede directamente a la base de datos: igual que la capa de presentación, solicita la información a los módulos de catálogo, inventario, pedidos y contenido, con lo que respeta la comunicación descendente definida en la arquitectura. La Figura 11 muestra el flujo conceptual.

**Figura 11**
*Flujo conceptual de integración del servicio de inteligencia artificial*




*Nota.* Las etiquetas numeradas sin apóstrofo corresponden al asistente de compra; las etiquetas con apóstrofo, al asistente de contenido. Por legibilidad no se dibuja el paso 5 del asistente de compra: el retorno de los resultados al modelo y la entrega de la respuesta final a la tienda. Elaboración propia.

En el asistente de compra, el flujo sería el siguiente: (1) el visitante escribe su pregunta en el widget de la tienda, que la envía a la API del sistema; (2) el módulo de asistencia la remite al modelo junto con instrucciones de comportamiento y la descripción de las funciones disponibles, como buscarProductos, consultarDisponibilidad o consultarEstadoPedido; (3) el modelo responde indicando qué función ejecutar y con qué parámetros; (4) el módulo ejecuta la consulta en el módulo correspondiente, aplicando los mismos permisos que a cualquier usuario —un visitante no puede consultar pedidos ajenos—; y (5) devuelve los resultados al modelo, que redacta la respuesta final con datos reales, y el módulo la entrega a la tienda. Este esquema corresponde a la generación aumentada por recuperación y a la llamada a funciones descritas en el marco conceptual (Lewis et al., 2020).

En el asistente de contenido, el administrador solicita un borrador desde la ficha del producto; el módulo envía al modelo los atributos registrados y la fotografía, y guarda el resultado en el módulo de contenido y metadatos con estado de borrador. El contenido solo se publica cuando el administrador lo aprueba, lo que mantiene a una persona como responsable de lo que se muestra al cliente.

Para preservar la seguridad y la privacidad, el módulo no enviaría al servicio externo datos personales de los clientes distintos de los estrictamente necesarios para la consulta, las credenciales de acceso a la API se almacenarían como secretos del servidor y nunca en el navegador, y cada interacción se registraría con su fecha para efectos de trazabilidad, en coherencia con RNF05 y RNF15.

### 13.4. Resultado esperado

Con la incorporación del servicio se espera beneficiar tanto al cliente como al negocio. Para el cliente, el beneficio es obtener respuestas inmediatas y orientación durante la compra, que se ampliará a medida que se definan los aspectos de apoyo que aún están en evaluación. Para el negocio, se espera que una parte significativa de las preguntas que hoy recibe el propietario —disponibilidad, tallas, materiales, precios y estado de pedidos— se resuelva de manera inmediata y a cualquier hora, reduciendo el tiempo que el propietario dedica a responder mensajes y evitando que se pierdan ventas por falta de respuesta. Del lado administrativo, se espera reducir el tiempo necesario para publicar una pieza nueva y lograr que todas las fichas del catálogo cuenten con descripción, metadatos y datos estructurados completos. La Tabla 19 resume el servicio propuesto.

**Tabla 19**

*Resumen del servicio de inteligencia artificial propuesto*

| Aspecto | Descripción |
|---|---|
| Servicio propuesto | API de Gemini en Vertex AI (Google Cloud), integrada mediante un adaptador intercambiable. |
| Función 1: asistente de compra | Responder en lenguaje natural preguntas sobre productos, precios, existencias y estado de pedidos, con datos reales obtenidos mediante llamada a funciones. |
| Apoyo al cliente (en evaluación) | Se proyecta que la IA ayude también al cliente; los aspectos concretos aún se están analizando (recomendaciones, orientación de tallas, explicación de materiales, cuidado de las piezas y acompañamiento posventa). |
| Función 2: asistente de contenido | Generar borradores de descripciones, metadatos, texto alternativo y datos estructurados a partir de los atributos y la fotografía de la pieza, sujetos a aprobación del administrador. |
| Problemas que atiende | Dependencia del contacto manual y falta de orientación al comprador fuera del horario de atención; catálogo multimaterial con riesgo de información incompleta; baja visibilidad de la oferta. |
| Integración | Nuevo módulo de asistencia con IA en la capa de lógica de negocio; consume los módulos existentes y no accede directamente a la base de datos. |
| Indicadores de resultado | Porcentaje de conversaciones resueltas sin intervención del propietario; tiempo promedio para publicar un producto; porcentaje de fichas con metadatos y datos estructurados completos. |
| Riesgos y controles | Respuestas incorrectas: se mitigan fundamentando las respuestas en datos del sistema y con revisión humana del contenido. Privacidad: se limita la información enviada. Costo: se establecen límites de uso. |
| Estado en esta entrega | Propuesta conceptual; no se implementa ni se conecta. |

*Nota.* Elaboración propia.

De esta manera, el servicio de inteligencia artificial no se plantea como un elemento decorativo, sino como un complemento directo del sistema transaccional: utiliza la información que el sistema ya registra para atender mejor a los compradores y para facilitar la administración del catálogo, y se integra respetando la arquitectura, los requerimientos de seguridad y la trazabilidad definidos a lo largo del documento.

## 14. Referencias

Aggarwal, P., Murahari, V., Rajpurohit, T., Kalyan, A., Narasimhan, K., & Deshpande, A. (2024). GEO: Generative engine optimization. En Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 5–16). Association for Computing Machinery. https://doi.org/10.1145/3637528.3671900

Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4.ª ed.). Addison-Wesley.

Booch, G., Rumbaugh, J., & Jacobson, I. (2006). El lenguaje unificado de modelado: Manual de referencia (2.ª ed.). Pearson Educación.

Cámara Colombiana de Comercio Electrónico. (2026). Informe del comportamiento del comercio electrónico en Colombia 2025. https://ccce.org.co/noticia/informes-ccce/

Date, C. J. (2001). Introducción a los sistemas de bases de datos (7.ª ed.). Pearson Educación.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Enge, E., Spencer, S., & Stricchiola, J. (2015). The art of SEO: Mastering search engine optimization (3.ª ed.). O'Reilly Media.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fowler, M. (2003). Patterns of enterprise application architecture. Addison-Wesley.

Google. (s.f.-a). Understanding Core Web Vitals and Google search results. Google Search Central. Recuperado el 18 de octubre de 2026, de https://developers.google.com/search/docs/appearance/core-web-vitals

Google. (s.f.-b). What is Angular? Angular. Recuperado el 18 de octubre de 2026, de https://angular.dev/overview

Google. (s.f.-c). Server-side and hybrid-rendering. Angular. Recuperado el 18 de octubre de 2026, de https://angular.dev/guide/ssr

Google Cloud. (s.f.). Grounding overview. Vertex AI Documentation. Recuperado el 18 de octubre de 2026, de https://docs.cloud.google.com/vertex-ai/generative-ai/docs/grounding/overview

International Organization for Standardization. (2023). Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model (ISO/IEC Standard No. 25010:2023). https://www.iso.org/standard/78176.html

Laudon, K. C., & Traver, C. G. (2021). E-commerce: Business, technology, society (16.ª ed.). Pearson.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

Ley 1480 de 2011. Por medio de la cual se expide el Estatuto del Consumidor y se dictan otras disposiciones. 12 de octubre de 2011. Diario Oficial No. 48.220.

Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales. 17 de octubre de 2012. Diario Oficial No. 48.587.

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1/

OWASP Foundation. (2025). OWASP Application Security Verification Standard (Versión 5.0.0). https://owasp.org/www-project-application-security-verification-standard/

PCI Security Standards Council. (2024). Payment Card Industry Data Security Standard: Requirements and testing procedures (Versión 4.0.1). https://www.pcisecuritystandards.org/

PostgreSQL Global Development Group. (s.f.). PostgreSQL documentation: Concurrency control. Recuperado el 18 de octubre de 2026, de https://www.postgresql.org/docs/current/mvcc.html

Pressman, R. S., & Maxim, B. R. (2021). Ingeniería del software: Un enfoque práctico (9.ª ed.). McGraw-Hill.

Schema.org. (2024). Product. https://schema.org/Product

Sommerville, I. (2011). Ingeniería de software (9.ª ed.). Pearson Educación.

Spring. (s.f.-a). Transaction management. Spring Framework Reference Documentation. Recuperado el 18 de octubre de 2026, de https://docs.spring.io/spring-framework/reference/data-access/transaction.html

Spring. (s.f.-b). Spring Boot. Spring Boot Reference Documentation. Recuperado el 18 de octubre de 2026, de https://docs.spring.io/spring-boot/index.html

Wompi. (s.f.). Eventos. Documentación Wompi Colombia. Recuperado el 18 de octubre de 2026, de https://docs.wompi.co/docs/colombia/eventos/

World Wide Web Consortium. (2023). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/